Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Fachbereich Informatik

Master's Thesis

Creating and Benchmarking Learned Database Components using LLMs

Author:
Preksha Rajesh Shah

submitted on:
April 16, 2026

Supervisor
Angjela Davitkova

Reviewers
Prof. Dr.-Ing. Sebastian Michel

Abstract

Lorem ipsum

Zusammenfassung

Lorem ipsum

Contents

1 Introduction **1**
1.1 Problem Statement . . . 1
1.2 Proposed Approach . . . 1
1.3 Contributions . . . 2

2 Background **3**
2.1 Learned Database Systems and Cardinality Estimation . . . 3
2.2 MSCN and Deep Learning-based Estimation . . . 3
2.3 Workload Sensitivity and Challenges . . . 4
2.4 LLM-based SQL Generation . . . 5
2.5 Generative AI for Database Components . . . 5
2.6 Active Learning for Data Efficiency . . . 6
2.7 Data Diversity, Personas, and KL Divergence . . . 6
2.8 System Perspective: Ollama and Practical Deployment . . . 7
2.9 Summary and Motivation . . . 7

3 Related Work **8**
3.1 Learned Cardinality Estimation . . . 8
3.2 Workload Characteristics and Model Generalization . . . 8
3.3 LLM-based SQL Workload Generation . . . 9
3.4 Generative AI for Training Database Components . . . 9
3.5 Active Learning for Efficient Data Utilization . . . 10
3.6 Data Diversity and Distribution Alignment . . . 10
3.7 Positioning of the Proposed Approach . . . 11

4 Approach **12**
4.1 Design Principles . . . 12
4.1.1 Schema Agnosticism . . . 12
4.1.2 Modularity and Extensibility . . . 12
4.1.3 Robustness and Fault Tolerance . . . 13
4.1.4 Reproducibility . . . 13
4.2 System Architecture Overview . . . 13
4.3 Pipeline Stages . . . 13
4.3.1 Stage 1: LLM-Based Query Generation . . . 14
4.3.2 Stage 2: Query Validation and Format Conversion . . . 15
4.3.3 Stage 3: Database Labeling . . . 15
4.3.4 Stage 4: Feature Extraction and Bitmap Generation . . . 16
4.3.5 Stage 5: Active Learning Loop . . . 16
4.3.6 Stage 6: Evaluation and Visualization . . . 17
4.4 Dataset and Experimental Configuration . . . 17
4.4.1 The IMDB Dataset . . . 17
4.4.2 Default Hyperparameters . . . 17
4.5 Workflow Execution . . . 19
4.6 Summary . . . 19

5 Query Generation Approach **21**
5.1 LLM-Based Query Generation . . . 21

5.1.1 Motivation for Using LLMs 21
5.1.2 LLM Selection and Configuration 21
5.2 Prompt Engineering Strategy 22
5.2.1 Prompt Template Structure 22
5.2.2 Schema Context Injection 22
5.2.3 Generation Constraint Rules 24
5.2.4 Diversity Hint Rotation 24
5.2.5 Output Format Specification 24
5.3 Batch Generation Strategy 25
5.3.1 Iterative Batch Processing 25
5.3.2 Retry Logic and Error Handling 25
5.4 Multi-Layer Validation Pipeline 27
5.4.1 Layer 1: Structural Validation 27
5.4.2 Layer 2: SQL Parsing and Clause Extraction 27
5.4.3 Layer 3: Schema Validation 28
5.4.4 Layer 4: Format Conversion 28
5.5 Synthetic Query Generator 28
5.5.1 Table and Column Definitions 30
5.5.2 Generation Algorithm 30
5.6 Query Workload Analysis 30
5.7 Comparison: LLM vs. Synthetic Generation 30
5.8 Summary 32

6 Training Approach **33**
6.1 Database Labeling Pipeline 33
6.1.1 SQL Reconstruction 33
6.1.2 Robust Query Execution 34
6.2 Materialized Samples and Bitmap Generation 35
6.2.1 Phase 1: Materialized Sample Creation 35
6.2.2 Phase 2: Runtime Bitmap Generation 35
6.3 Query Feature Encoding 36
6.3.1 Vocabulary Construction 37
6.3.2 Per-Component Encoding 37
6.3.3 Padding and Masking 37
6.4 Multi-Set Convolutional Network (MSCN) 38
6.4.1 Architecture Overview 38
6.4.2 Set Processing Modules 38
6.4.3 Masked Mean Pooling 38
6.4.4 Output Network 38
6.4.5 Label Normalization 40
6.5 Loss Function: Q-Error 40
6.6 Active Learning Framework 41
6.6.1 Problem Formulation 41
6.6.2 Acquisition Strategies 41
6.6.3 Active Learning Loop 42
6.6.4 Label Budget Analysis 42
6.7 Evaluation Framework 42
6.7.1 Metrics 42
6.7.2 Learning Curve Visualization 44
6.7.3 Predicted vs. Actual Scatter Plot 44

6.7.4 Training Loss Across Rounds 44
6.8 Q-Error Analysis Across Rounds 44
6.8.1 Per-Round Q-Error Statistics 44
6.8.2 Sample Efficiency 44
6.9 Summary 45

7 Experimental Results 46
7.1 Experimental Setup 46
7.1.1 Dataset 46
7.1.2 Configuration 46
7.2 Query Generation Analysis 46
7.2.1 Structural Feature Distributions 47
7.2.2 Cardinality Distribution 47
7.3 Active Learning and Model Training Results 47
7.3.1 Learning Curves 48
7.3.2 Training Loss Across Rounds 48
7.3.3 Predicted vs. Actual Cardinality 48
7.3.4 Q-Error Distribution 48
7.4 Pipeline Summary 48
7.5 Discussion 48

8 Conclusion 52
8.1 Summary of Contributions 52
8.2 Impact and Implications 52
8.2.1 Database Systems Research 52
8.2.2 Practical Applications 52
8.2.3 Methodological Advances 53
8.3 Limitations and Future Work 53
8.3.1 Current Limitations 53
8.3.2 Future Research Directions 53
8.4 Final Remarks 54

List of Figures

4.1 High-level system architecture showing the end-to-end pipeline from schema ingestion through query generation, validation, labeling, and active learning to final model evaluation. The dashed feedback loop indicates iterative refinement through multiple active learning rounds. 14

4.2 Entity-relationship diagram of the IMDB database schema used in experiments. Primary keys (PK) and foreign keys (FK) are annotated. The `title_principals` table serves as a junction table connecting titles to people. 18

5.1 Structure of the LLM prompt template, consisting of five hierarchically organized components that guide the model from context understanding to constrained output generation. 23

5.2 Multi-layer validation pipeline for LLM-generated queries. Each layer progressively filters invalid queries, with rejected queries logged for analysis. Valid queries proceed to the MSCN format conversion stage. 29

6.1 Complete MSCN architecture. Each query component (samples, predicates, joins) passes through a dedicated two-layer MLP with dropout, followed by masked mean pooling. The pooled representations are concatenated and processed through an output MLP with sigmoid activation to produce a normalized cardinality estimate $\hat{y} \in [0, 1]$. 39

7.1 Combined structural feature distributions of the generated query workload. Left: number of tables per query. Center: number of join conditions per query. Right: number of filter predicates per query. 47

7.2 Distribution of the number of tables per generated query. 47

7.3 Distribution of the number of filter predicates per query. 47

7.4 Distribution of true cardinalities for labeled queries ($\log_{10}$ scale). The distribution spans from single-digit cardinalities to tens of millions of rows, ensuring comprehensive coverage of the selectivity spectrum. 48

7.5 Active learning curve showing validation median Q-error versus number of labeled queries. The decreasing trend demonstrates that the model's estimation accuracy improves as more informative queries are labeled and added to the training set in each active learning round. 49

7.6 Training loss curves across active learning rounds. The decreasing loss trajectory across rounds indicates that the model benefits from the progressively expanding and more informative training set. 49

7.7 Predicted vs. actual cardinality for the final trained model (log-log scale). Points clustering around the diagonal indicate accurate predictions across the full cardinality range. 50

7.8 Cumulative distribution function (CDF) of Q-errors for the final model. 50
7.9 Q-error box plots per active learning round showing improving accuracy. 50
7.10 Pipeline summary dashboard showing key metrics from query generation, labeling, training, and evaluation across all active learning rounds. 51

List of Algorithms

1 Complete Pipeline Execution Workflow 20
2 LLM Batch Query Generation with Retry Logic 26
3 Structural SQL Validation 27
4 Synthetic Query Generation 31
5 SQL Reconstruction from Structured Query Components 33
6 Robust Query Labeling with Timeout and Error Recovery 34
7 Materialized Sample Creation 35
8 Runtime Bitmap Generation for a Single Query 36
9 Complete Active Learning Loop with Database Labeling 43

List of Tables

4.1 Default hyperparameters for the complete pipeline. 18

5.1 Synthetic generator table definitions with column ranges. 30
5.2 Comparison of LLM-based and synthetic query generation approaches. 31

6.1 Q-error statistics across active learning rounds for different acquisition strategies (lower is better). 45

7.1 Experimental configuration parameters. 46

Chapter 1

Introduction

1.1 Problem Statement

Modern database systems rely heavily on cardinality estimation to optimize query execution. Accurate estimation of intermediate result sizes is essential for selecting efficient execution plans. However, traditional cardinality estimators are based on simplifying assumptions such as attribute independence and uniform data distributions, which rarely hold in real-world datasets. As a result, these estimators often produce inaccurate predictions, leading to suboptimal query plans and degraded system performance.

Recent advances in learned database systems have demonstrated that machine learning models can replace traditional components and achieve significantly improved estimation accuracy. In particular, the Multi-Set Convolutional Network (MSCN) shows that deep learning models can capture complex correlations across tables, joins, and predicates [4]. Despite these improvements, learned approaches introduce a new challenge: they require large volumes of labeled training data, where each label is obtained by executing a query on a database system. This process is computationally expensive, time-consuming, and difficult to scale.

At the same time, large language models (LLMs) have shown strong capabilities in generating structured data, particularly SQL queries. Frameworks such as SQLStorm demonstrate that LLMs can generate diverse and realistic SQL workloads at scale [11]. However, existing approaches still rely on executing these queries to obtain ground truth labels, leaving the fundamental bottleneck of execution-based labeling unresolved.

Therefore, a key open problem is how to generate large-scale, diverse, and labeled training data for learned database components without relying on expensive database execution.

1.2 Proposed Approach

In this thesis, we propose a novel perspective on the role of large language models in database systems. Instead of using LLMs during query execution, we treat them as data generators. Specifically, an LLM is used to generate synthetic SQL queries along with their corresponding labels or training signals. These query–label pairs are then used to train a supervised model that implements a database component, such as a learned cardinality estimator. The trained model, rather than the LLM, is used during runtime.

To address the challenge of data efficiency, we integrate active learning into the training process. Instead of labeling all generated queries, the model selectively

identifies the most informative queries based on uncertainty and diversity criteria. Only these selected queries are labeled and incorporated into the training dataset, significantly reducing the cost of data generation while maintaining high model performance.

The proposed approach is implemented as a fully automated, schema-agnostic pipeline that integrates query generation, validation, feature extraction, active learning, and model training. The pipeline supports both LLM-based and synthetic query generation and can operate on arbitrary database schemas without manual customization. This enables scalable experimentation and efficient training of learned database components.

Our approach is motivated by prior observations that workload diversity and query characteristics play a crucial role in the performance of learned models [12]. By leveraging LLMs to generate diverse workloads and combining this with active learning, we aim to improve the robustness and generalization of learned cardinality estimators while reducing reliance on execution-based labeling.

1.3 Contributions

The main contributions of this thesis are as follows:

- We propose a novel framework that leverages large language models to generate synthetic SQL queries and their corresponding training signals, reducing dependence on manually collected query workloads.
- We introduce an active learning-based training strategy for learned cardinality estimation, enabling efficient selection of informative queries and significantly reducing labeling costs.
- We design and implement a fully automated, schema-agnostic pipeline that integrates query generation, validation, feature extraction, active learning, and model training.
- We demonstrate how combining LLM-based data generation with active learning improves scalability, diversity, and effectiveness of training data for learned database components.

Chapter 2

Background

2.1 Learned Database Systems and Cardinality Estimation

Modern database systems use query optimizers to optimize queries for efficient execution. At the heart of query optimization is cardinality estimation. Cardinality estimation is used to predict the number of tuples that are generated during intermediate steps of query execution. The estimates are used to optimize queries and improve performance. However, small errors in cardinality estimation can lead to poor query optimization and performance. Conventional cardinality estimation techniques use statistical estimates such as histograms and sampling. Other techniques include making assumptions such as attribute independence and uniform distribution of values. Although conventional techniques are computationally efficient, they do not reflect real-world data characteristics. Real-world data has complex correlations among attributes and tables, especially for join-heavy queries. As such, conventional techniques often yield poor estimates for queries with multiple predicates and joins.

In addition to inaccuracies caused by independence assumptions, another limitation of traditional approaches lies in their inability to adapt dynamically to evolving data distributions. In real-world applications, data is continuously updated, and query workloads may change over time. Static statistical summaries often fail to capture these changes effectively, leading to outdated estimates and suboptimal execution plans. This problem becomes more severe in large-scale systems where maintaining accurate statistics is itself a costly operation. As a result, there has been a growing interest in approaches that can learn directly from data and adapt to changing workloads.

This limitation has motivated the exploration of learned database systems, where machine learning models replace traditional components. Instead of relying on predefined assumptions, these models learn patterns directly from data, enabling them to capture correlations that are otherwise difficult to model. Learned cardinality estimation is one of the most actively studied areas within this paradigm. These models aim to generalize across different query structures and data distributions, providing more robust and accurate estimates compared to traditional techniques.

2.2 MSCN and Deep Learning-based Estimation

One important advancement in the area of learning-based query cardinality estimation is the development of a new technique called Multi-Set Convolutional Network (MSCN), proposed by Kipf et al. in [4]. The technique reformulates query cardinality estimation as a supervised learning task, where query result

sizes are estimated based on query structures.

The basic idea behind the MSCN technique is that it represents queries in the form of sets. Unlike traditional methods, where a query is simply represented as a string of characters, MSCN represents a query as three different sets, namely tables, joins, and predicates. Each set is processed separately using shared layers of a neural network. The basic idea behind this is to make the query invariant to the ordering of query components, which is important because the semantics of a query do not depend on the order in which tables and predicates are listed.

The other key feature of the MSCN is its use of bitmap-based sampling. In the process, a subset of the tuples in a given base table is sampled, and the predicates are applied to the sample to produce bitmaps. The bitmaps then represent the query predicates that are satisfied, and this additional information helps the model to better account for the correlations between the predicates and the data.

In addition, the MSCN uses normalization to ensure that the cardinality estimates are within a certain range that can be learned by the model. This is crucial, especially considering that the cardinality estimates can differ considerably across queries.The MSCN uses loss functions that involve relative errors, as opposed to absolute errors, which is more appropriate for query optimization tasks.

Despite its strong performance, MSCN depends heavily on labeled training data. Each training example requires a query and its true cardinality, which is obtained by executing the query on a database. As query complexity increases, the cost of execution grows significantly, making it difficult to scale the training process. This limitation highlights a fundamental challenge in learned database systems: the high cost of data generation. In addition, the reliance on execution makes it difficult to quickly generate new training data when schemas or workloads change.

2.3 Workload Sensitivity and Challenges

The performance of learned cardinality estimators not only depends on the architecture used but also on the nature and characteristics of the training workload. The study conducted by Wang et al. [12] offers a comprehensive analysis on how different query properties influence the performance of learned cardinality estimators.

For example, a learned model may not perform well when it is tested on queries that have different types of predicates, such as range predicates, when it is only trained on queries that have equality predicates. Similarly, a query may have high selectivity and join complexity, causing a large estimation error. This example again emphasizes that the training data should be highly diverse.

Another important aspect to be considered is related to workload shift, where the query distribution is different from the training distribution. In this case, learned models may not perform well, and this again emphasizes that the training data should be highly diverse. Moreover, the interaction between multiple

query features, such as the number of joins and predicate selectivity, can further complicate the estimation process. These interactions are often difficult to model explicitly, which reinforces the importance of exposing models to a wide variety of query patterns during training.

2.4 LLM-based SQL Generation

Recently, large language models have been recognized as effective means for generating structured data. In this regard, the SQLStorm framework [11] has been presented to show that LLMs can be effectively used to produce large volumes of varied and realistic SQL queries. In this regard, the SQLStorm framework exploits the capabilities of schema-informed prompting to produce varied and realistic queries.

Unlike other query generation techniques that are based on template-based and rule-based approaches, LLMs can be effectively used to produce queries that can capture a wide range of query patterns. This is one of the main reasons that LLMs can be effectively used to produce training datasets for learning-based query generation techniques. In addition to this, LLMs can be effectively used to produce queries that are not included in existing workloads. This allows for the exploration of new query patterns.

Another important advantage of LLM-based generation is its ability to incorporate semantic understanding of schemas. By conditioning on table and column descriptions, LLMs can generate queries that are not only syntactically correct but also semantically meaningful. This capability is particularly useful when dealing with complex schemas, where manual query generation becomes challenging.

Although LLMs can be effectively used to produce queries, they do not have the capability to produce the labels that are associated with the queries. This means that the produced queries have to be executed to produce labels. This is one of the main drawbacks of using LLMs for query generation, as it reintroduces the dependency on database execution.

2.5 Generative AI for Database Components

The base paper [1] attempts to address this data generation bottleneck through a framework that utilizes generative models to automatically generate training data for learned database components. The basic idea is to utilize LLMs to automatically generate realistic SQL queries, thereby reducing reliance on manually collected query logs. This is a major departure from the traditional way learned database systems have been designed. By automating query generation, it is possible to generate large and varied datasets to support a wide range of query types. The framework also outlines how generative models can be incorporated into the learning pipeline to support data generation.

An important contribution of this work is the structured pipeline it proposes, where query generation, validation, execution, and model training are integrated into a unified framework. This pipeline ensures that generated queries are not

only diverse but also executable and meaningful within the given schema. The use of prompt engineering and iterative refinement further improves the quality of generated queries.

One limitation of this framework is that it relies on executing the generated queries to obtain ground truth labels. While query generation is now automated, label generation is computationally intensive and thus limits scalability. This is where a completely synthetic method is proposed, where query and label generation can be performed without relying on database execution. This shift represents a natural evolution of the framework, aiming to remove the remaining bottleneck in the data generation process.

2.6 Active Learning for Data Efficiency

Active learning helps in developing a framework for improving data efficiency, which can be done through the selection of the best data samples for learning. As discussed in [5], not all data samples are equally important in improving the performance of the model. This way, it is possible to attain high accuracy with less data.

In the context of LQE using learned cardinality estimation, active learning helps in improving the efficiency of the process of query labeling. Instead of labeling all the generated queries, the model can be improved through the selection of the best queries in improving the performance of the model. This selection of the best queries can be done using the uncertainty of the model.

In addition to uncertainty-based selection, active learning can also incorporate diversity-based sampling, where queries that represent different regions of the query space are selected. This ensures that the model is exposed to a wide range of patterns while avoiding redundancy in the training data. Such strategies are particularly useful when working with large volumes of synthetically generated data.

The integration of active learning with the process of synthetic data generation helps in improving the efficiency of the learning process. LLMs are capable of generating a wide variety of data samples in the form of queries. Active learning helps in the selection of the best data samples, thus improving the efficiency of the learning process.

2.7 Data Diversity, Personas, and KL Divergence

Diversity in the generated data is a critical factor to be taken into consideration for the training of strong models. The PAARS [7] framework proposes the use of personas for generating diverse and realistic data. Personas help in the simulation of diverse behavior, which enables the generation of diverse data.

In order to test the quality of the generated data, the PAARS framework uses the KL divergence as a similarity metric between the generated data and real-world data. The KL divergence is a measure of the dissimilarity between two probability distributions, which can be used to test the similarity between the generated data and real-world data, especially in the case of SQL queries.

In the context of query generation, KL divergence can be applied to distributions over query features such as the number of joins, predicate types, and selectivity ranges. By comparing these distributions between synthetic and real workloads, it is possible to evaluate whether the generated data accurately reflects real-world patterns. This provides a quantitative basis for improving the quality of synthetic datasets.

2.8 System Perspective: Ollama and Practical Deployment

In terms of system efficiency, the application of LLMs for data generation also requires careful consideration. In this work, Ollama is used to locally run LLMs. This allows for efficient query generation. Local running also provides several advantages, including efficiency, privacy, and ease of experimentation.

Another advantage of using local running for LLMs is the ability to control the models. This provides reproducibility of experiments. Additionally, this approach helps to ensure that the results are not affected by external APIs. Local running also provides the advantage of fine-tuning models based on specific requirements.

The application of Ollama provides the advantage of generating queries. This process can be refined and validated. This setup provides the advantage of ensuring that the generated data meets the requirements for the training process.

2.9 Summary and Motivation

The background provided in this section points to the evolution of learned database systems, starting with conventional estimators, followed by machine learning models, and then moving on to the latest generative models. While the MSCN [4] paper proves the efficiency of learned models, it also points to the disadvantage of data generation. Subsequent papers focus on the need for diversity in the workload [12] and the application of LLMs in query generation [11].

However, the conventional methods are all based on execution-based labeling, which is not scalable. This thesis, with the help of insights provided by generative models, active learning [5], and diversity-aware models, proposes the application of LLMs in generating both the query and the label, thus using the learned model as the final database component.

Chapter 3

Related Work

3.1 Learned Cardinality Estimation

The use of machine learning to replace traditional database components has gained significant attention, particularly in the context of cardinality estimation. A foundational contribution in this area is the Multi-Set Convolutional Network (MSCN) proposed by Kipf et al. [4], which reformulates cardinality estimation as a supervised learning problem. Unlike conventional estimators that rely on simplifying assumptions such as attribute independence, MSCN represents queries as sets of tables, joins, and predicates, allowing the model to capture complex correlations across query components.

A key strength of MSCN lies in its ability to model join-crossing correlations, which are difficult for traditional estimators to handle effectively. Furthermore, the use of bitmap-based sampling enables the model to incorporate information about intermediate query results, improving estimation accuracy for complex queries involving multiple joins and predicates.

Despite these advantages, MSCN fundamentally depends on execution-based labeling. Each training instance requires executing a query on a database to obtain its true cardinality. This introduces substantial computational overhead and limits scalability, particularly when large and diverse workloads are required. Moreover, adapting to new schemas or workloads necessitates additional query execution, further increasing the cost. These limitations highlight a central challenge in learned cardinality estimation: the dependency on labeled data generated through expensive query execution.

3.2 Workload Characteristics and Model Generalization

Beyond model architecture, the effectiveness of learned cardinality estimators is strongly influenced by the characteristics of the training workload. Wang et al. [12] provide a comprehensive analysis demonstrating that learned models are highly sensitive to factors such as predicate types, selectivity, join complexity, and query structure.

A key observation is that models trained on narrow or biased workloads often fail to generalize to unseen queries. For example, a model trained primarily on equality predicates may perform poorly when evaluated on range predicates or more complex join conditions. Similarly, variations in selectivity can significantly impact estimation accuracy, as models struggle to extrapolate beyond observed distributions.

The issue of workload shift further complicates deployment. In real-world sys-

tems, query distributions evolve over time, leading to discrepancies between training and inference workloads. Under such conditions, learned models can exhibit significant performance degradation. These findings emphasize the importance of constructing training datasets that are both diverse and representative of real-world query patterns.

Recent work on robust cardinality estimation under changing workloads [8] further highlights the importance of adaptability, reinforcing the need for data generation approaches that can capture a wide range of query characteristics.

3.3 LLM-based SQL Workload Generation

The emergence of large language models (LLMs) has introduced new possibilities for SQL workload generation. The SQLStorm framework [11] demonstrates that LLMs can generate large-scale, diverse, and realistic SQL queries using schema-aware prompting techniques.

Unlike traditional template-based approaches, which are inherently limited in diversity, LLM-based methods can generate queries with varying levels of complexity, including different join structures, predicate combinations, and nested queries. This flexibility enables the generation of workloads that better reflect real-world usage patterns.

Additional work in text-to-SQL and SQL understanding, such as CHASE-SQL [9] and evaluation studies by Rahaman et al. [10], further demonstrate the capability of LLMs to understand and generate structured queries. Similarly, TAPEX [6] explores pre-training techniques for table reasoning, highlighting the broader applicability of neural models in structured data tasks.

However, existing LLM-based generation approaches primarily focus on query generation rather than labeled data creation. Generated queries still require execution to obtain ground truth cardinalities, thereby retaining the dependency on database systems and limiting scalability for training purposes.

3.4 Generative AI for Training Database Components

Recent work on integrating generative AI into database systems has sought to address the limitations of manual and execution-based data generation. Davitkova and Michel [1] propose a framework that leverages generative models to automate the training of learned database components.

This framework introduces a structured pipeline that integrates query generation, validation, execution, and model training. By automating query generation, it significantly reduces reliance on manually collected query logs and enables the creation of diverse training datasets. Techniques such as prompt engineering and iterative refinement are used to improve the quality and realism of generated queries.

Despite these advancements, the framework still depends on executing queries to obtain ground truth labels. While generation is automated, labeling remains

computationally expensive, limiting scalability. This highlights a critical gap: existing approaches have automated query generation but have not eliminated the dependency on execution-based labeling.

3.5 Active Learning for Efficient Data Utilization

Active learning provides a complementary approach to improving data efficiency in supervised learning systems. Recent advances in deep active learning [5] show that selecting informative samples can significantly reduce the amount of labeled data required for training.

In the context of learned cardinality estimation, active learning can be used to minimize the number of queries that need to be executed for labeling. By prioritizing queries where the model is uncertain or performs poorly, it is possible to achieve high accuracy with fewer labeled samples.

In addition to uncertainty-based strategies, diversity-based sampling plays an important role in ensuring that selected queries cover a wide range of patterns. This is particularly relevant when working with synthetically generated data, where redundancy may otherwise reduce training effectiveness. Combining uncertainty and diversity criteria allows for the construction of efficient and representative training datasets.

Although active learning has been extensively studied in machine learning, its application to learned database systems remains relatively limited. Integrating active learning with LLM-based data generation presents a promising direction for improving both scalability and efficiency.

3.6 Data Diversity and Distribution Alignment

Ensuring that synthetic data reflects real-world distributions is a critical challenge. The PAARS framework [7] introduces persona-based generation to simulate diverse user behaviors, enabling the creation of varied and realistic data.

A key contribution of this work is the use of KL divergence to measure the similarity between generated and real data distributions. This provides a quantitative method for evaluating whether synthetic data accurately captures real-world characteristics. In the context of SQL workloads, such metrics can be applied to distributions over query features, including join counts, predicate types, and selectivity.

Distribution alignment is essential for training robust models. If synthetic data does not reflect real-world usage patterns, learned models may fail to generalize effectively. Incorporating distributional metrics into the generation process helps ensure that synthetic datasets are both diverse and representative.

3.7 Positioning of the Proposed Approach

The existing literature demonstrates significant progress in learned database systems and LLM-based data generation. Models such as MSCN [4] highlight the potential of learned approaches for cardinality estimation, while studies on workload characteristics [12] emphasize the importance of diverse training data. LLM-based frameworks such as SQLStorm [11] and generative pipelines [1] demonstrate the feasibility of large-scale query generation.

However, a common limitation across these approaches is the continued reliance on execution-based labeling. While query generation has been largely automated, obtaining ground truth labels still requires executing queries on a database, introducing significant computational overhead.

The approach proposed in this thesis addresses this limitation by leveraging LLMs to generate both queries and their corresponding labels, thereby eliminating the need for database execution during training. Furthermore, by incorporating active learning [5] and diversity-aware generation strategies [7], the proposed method aims to improve both data efficiency and representativeness.

In this framework, the LLM serves as a data generator, while the trained supervised model acts as the final database component during inference. This separation enables scalable and efficient training pipelines, representing a step toward fully automated learned database systems.

Chapter 4

Approach

This chapter presents the comprehensive approach for creating and benchmarking learned database components using large language models (LLMs) and active learning. We introduce the overall system architecture and describe each pipeline stage at a high level, with the detailed treatment of the query generation and training approaches deferred to subsequent chapters. Experimental results including workload analysis, learning curves, and model evaluation are presented in Chapter 7.

4.1 Design Principles

The design of our framework is guided by several key principles that ensure practical applicability, scalability, and extensibility:

4.1.1 Schema Agnosticism

The framework is designed to work with arbitrary relational database schemas without manual configuration. Given a schema definition (table names, column names, data types, and foreign key relationships), the system automatically generates appropriate queries, validates them against the schema, and produces correctly structured training data. This is achieved through:

- Dynamic schema parsing that extracts table metadata and relationships
- Runtime column statistics extraction (min/max values) for realistic predicate generation
- Automatic primary key detection for materialized sample creation
- Schema-aware query validation that rejects references to non-existent tables or columns

4.1.2 Modularity and Extensibility

The system is organized into modular components that can be independently modified or replaced:

- **Query Generation Module**: Currently uses Ollama-based LLMs but can be extended to any LLM provider or template-based generator.
- **Active Learning Module**: Supports pluggable acquisition functions (random, uncertainty, MC dropout), with a well-defined interface for adding new strategies.
- **Model Architecture**: While MSCN is the default learned estimator, the training pipeline can accommodate other architectures.

- **Database Backend**: Although developed and tested with PostgreSQL and the IMDB dataset, the pipeline is compatible with any JDBC/ODBC-accessible database.

4.1.3 Robustness and Fault Tolerance

Production-grade robustness is ensured through several mechanisms:

- Timeout handling for long-running database queries with configurable thresholds
- Automatic retry logic with exponential backoff for transient database errors
- Graceful degradation: queries that fail after maximum retries are assigned a fallback cardinality of 1
- Transaction management with proper commit/rollback semantics at every stage
- Comprehensive logging of failures for post-hoc analysis

4.1.4 Reproducibility

Scientific rigor requires reproducible experiments. The framework supports:

- Configurable random seeds for deterministic behavior across all random operations (Python, NumPy, PyTorch)
- Complete experiment configuration logging (all hyperparameters are saved to disk)
- CSV-based results export for downstream analysis
- Timestamped output directories to prevent overwriting previous experiments

4.2 System Architecture Overview

The proposed system consists of an end-to-end pipeline that integrates six major stages: (1) query generation, (2) query validation and format conversion, (3) database labeling, (4) feature extraction and bitmap generation, (5) active learning loop with model training, and (6) evaluation and visualization. Figure 4.1 illustrates the high-level architecture of the complete system.

4.3 Pipeline Stages

This section provides a high-level description of each pipeline stage. Detailed algorithmic treatments of the query generation and training approaches are presented in Chapters 5 and 6, respectively.

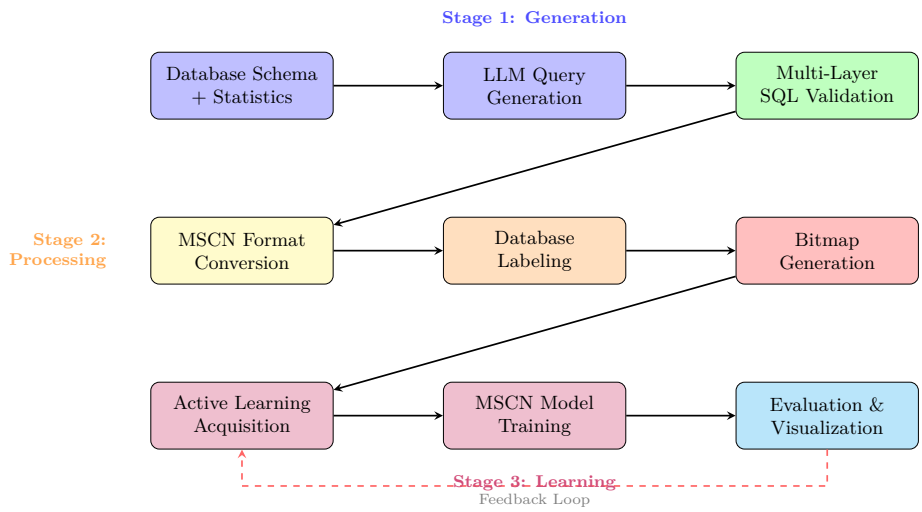

Figure 4.1: High-level system architecture showing the end-to-end pipeline from schema ingestion through query generation, validation, labeling, and active learning to final model evaluation. The dashed feedback loop indicates iterative refinement through multiple active learning rounds.

4.3.1 Stage 1: LLM-Based Query Generation

The first stage generates a large pool of unlabeled SQL queries by leveraging large language models. Unlike traditional synthetic query generators that rely on predefined templates or random sampling from the schema, LLMs capture complex linguistic and structural patterns present in real-world query workloads. The generation process is parameterized by:

- **Database Schema**: Complete DDL statements defining tables, columns, data types, and foreign key relationships fed as context to the LLM prompt.
- **Column Statistics**: Optional min/max values for numeric columns, enabling the LLM to generate predicates with realistic value ranges.
- **Diversity Hints**: Rotating textual hints that steer the LLM toward generating queries of varying complexity (single-table, 2-table joins, 3-table joins) and selectivity profiles.
- **Batch Size**: Queries are generated in configurable batches (default: 20 per LLM call) to enable progress tracking and partial failure recovery.

The system communicates with the LLM through the Ollama API, supporting any locally hosted model (e.g., Llama 3.2). The temperature parameter controls generation randomness, with higher values producing more diverse but potentially less valid queries.

4.3.2 Stage 2: Query Validation and Format Conversion

Raw LLM output undergoes rigorous multi-layer validation before entering the pipeline. This is critical because LLMs, while powerful, can produce syntactically or semantically invalid SQL. The validation pipeline consists of four sequential layers:

1. **Structural Validation**: Regex-based filtering rejects queries containing forbidden SQL constructs (OR, IN, LIKE, BETWEEN, subqueries, aggregations) that fall outside the supported query fragment.
2. **Syntax Parsing**: Each query is parsed to extract FROM, WHERE, and JOIN clauses, verifying correct SQL structure.
3. **Schema Validation**: All table names, column names, and aliases referenced in the query are verified against the provided schema definition.
4. **Format Conversion**: Valid queries are converted from raw SQL strings into the structured MSCN-compatible format, decomposing each query into its three constituent sets: tables, joins, and predicates.

The MSCN format represents each query as a CSV line with four hash-separated fields:

Listing 4.1: MSCN CSV Format Example

```
tables#joins#predicates#cardinality
title t,movie_info mi#t.id=mi.movie_id#t.kind_id,=,7,mi.info_type_id,>,16#12345
```

This compact representation enables efficient storage, parsing, and batch processing of large query workloads.

4.3.3 Stage 3: Database Labeling

Labeling assigns ground truth cardinalities to queries by executing them against the target database. This is the most computationally expensive stage, motivating the use of active learning to minimize the number of queries that need to be executed.

The labeling pipeline implements robust error handling:

- **SQL Reconstruction**: Each structured query is reconstructed into executable `SELECT COUNT(*)` SQL by reassembling tables, join conditions, and filter predicates.
- **Timeout Management**: Per-query timeouts (default: 6000 ms) prevent long-running queries from blocking the pipeline. Queries that exceed the timeout are retried up to a configurable maximum.
- **Error Recovery**: Schema errors (e.g., missing columns due to schema evolution) and other database errors are caught and logged, with affected queries assigned a fallback cardinality.
- **Minimum Cardinality**: All cardinalities are clamped to a minimum of 1 to satisfy MSCN's requirement for positive, log-normalizable values.

4.3.4 Stage 4: Feature Extraction and Bitmap Generation

The MSCN architecture requires fixed-size feature vectors representing both the query structure and the underlying data distribution. Feature extraction produces three types of encodings:

1. **Table Encoding**: Each table is represented as a one-hot vector concatenated with a bitmap vector. The bitmap captures the selectivity of the query's predicates on a pre-sampled set of rows from that table.
2. **Predicate Encoding**: Each predicate is represented as the concatenation of a column one-hot vector, an operator one-hot vector, and a normalized predicate value.
3. **Join Encoding**: Each join condition is represented as a one-hot vector over the set of all observed join patterns.

Bitmap generation is performed using materialized samples—a fixed set of randomly sampled primary key values drawn once at pipeline initialization. For each query, the system evaluates which sampled rows satisfy the query's predicates, producing a binary vector per table. This approach amortizes the sampling cost across all queries while providing an efficient approximation of the data distribution visible to each query.

4.3.5 Stage 5: Active Learning Loop

The active learning loop is the core optimization mechanism that reduces labeling cost. It operates iteratively:

1. An initial batch of queries is randomly selected from the pool and labeled.
2. The MSCN model is trained on the current labeled set.
3. The model is evaluated on a held-out validation set.
4. An acquisition function scores all unlabeled queries based on their estimated informativeness.
5. The top-scoring queries are selected, labeled, and added to the training set.
6. Steps 2–5 repeat for a configured number of rounds.

The framework supports three acquisition strategies:

- **Random Sampling**: Uniformly random selection, serving as the baseline.
- **Uncertainty Sampling**: Selects queries where the model's normalized prediction is closest to 0.5, indicating maximum uncertainty.
- **MC Dropout**: Performs $T = 25$ stochastic forward passes with dropout enabled, selecting queries with highest prediction variance as a Bayesian approximation of model uncertainty.

4.3.6 Stage 6: Evaluation and Visualization

The final stage produces comprehensive evaluation metrics and visualizations:

- **Q-Error Metrics**: Median, 90th percentile, and 95th percentile Q-error on the validation set, computed after each active learning round.
- **Learning Curves**: Plots of validation Q-error versus the number of labeled training queries, demonstrating sample efficiency.
- **Training Loss Curves**: Per-round and per-epoch training loss trajectories.
- **Predicted vs. Actual Scatter**: Log-log scatter plot comparing model predictions to ground truth cardinalities.
- **Q-Error CDF**: Cumulative distribution function of Q-errors with percentile annotations.
- **Workload Analysis**: Distributions of tables, joins, and predicates per query, along with cardinality distributions and labeling success rates.

The evaluation module generates 15 diagnostic plots organized into three categories (data generation analysis, model training and testing, and analysis summary), providing a comprehensive dashboard for experiment analysis.

4.4 Dataset and Experimental Configuration

4.4.1 The IMDB Dataset

All experiments in this thesis are conducted on the Internet Movie Database (IMDB), a widely used benchmark for cardinality estimation research. The IMDB dataset used in our experiments consists of four core tables:

- **title_basics**: Contains metadata for approximately 10 million titles, including title type, production year, runtime, and genre information.
- **name_basics**: Contains metadata for approximately 12 million people associated with titles, including name, birth year, and death year.
- **title_principals**: A junction table linking titles to the principal cast and crew, with approximately 56 million records.
- **title_ratings**: Contains user ratings and vote counts for approximately 1.3 million titles.

The schema defines foreign key relationships between tables through the `tconst` and `nconst` identifiers, enabling meaningful join queries. Figure 4.2 illustrates the schema relationships.

4.4.2 Default Hyperparameters

Table 4.1 summarizes the default hyperparameters used across all experiments unless otherwise specified. These values were selected based on preliminary experiments and align with recommended settings from the MSCN literature.

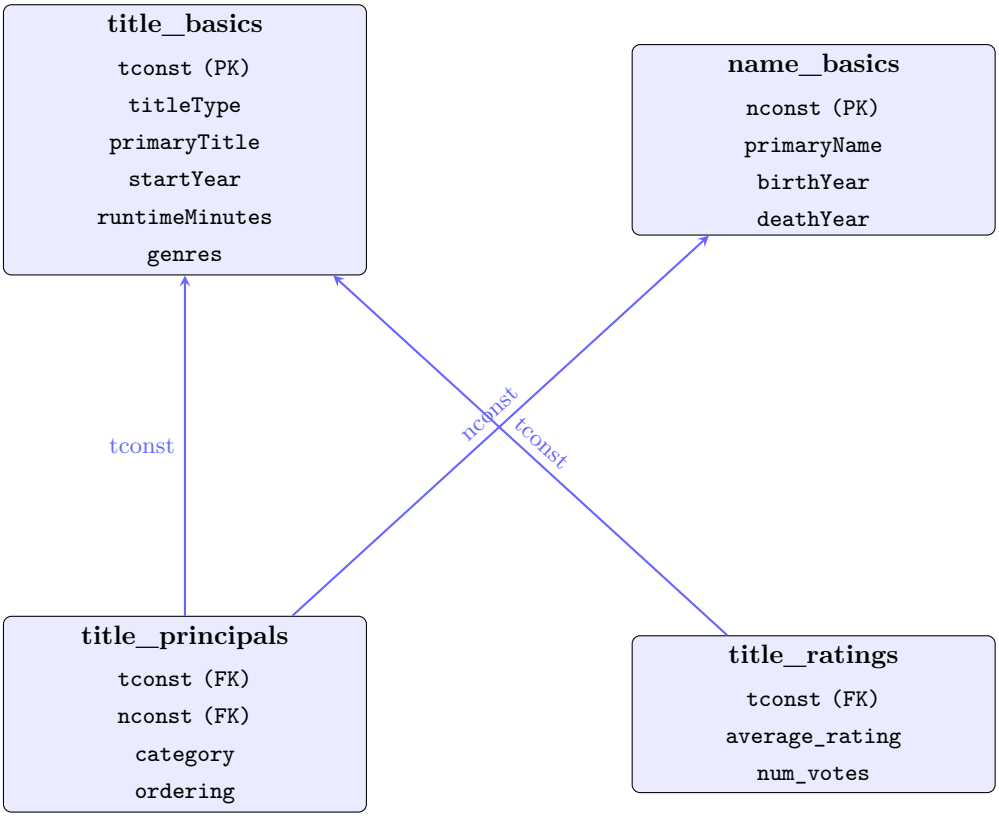

Figure 4.2: Entity-relationship diagram of the IMDB database schema used in experiments. Primary keys (PK) and foreign keys (FK) are annotated. The `title_principals` table serves as a junction table connecting titles to people.

Table 4.1: Default hyperparameters for the complete pipeline.

Category	**Parameter**	**Default Value**
Query Generation	Total queries	5000
	LLM batch size	20
	LLM model	Llama 3.2
	Temperature	0.8
Database	Query timeout	6000 ms
	Max retries	2
	Materialized samples	1000
Active Learning	Strategy	random / uncertainty / mc_dropout
	Rounds	5
	Acquisition batch size	200
	MC Dropout passes (T)	25
	Validation split	10%
MSCN Training	Hidden units	256
	Learning rate	10^{-4}
	Epochs per round	10
	Training batch size	1024

4.5 Workflow Execution

The complete pipeline is orchestrated by a single entry point (`pipeline.py`) that accepts all configuration parameters via command-line arguments or environment variables. Algorithm 1 presents the high-level execution workflow.

4.6 Summary

This chapter introduced the overarching approach for creating and benchmarking learned database components using LLMs and active learning. We motivated the problem of training data acquisition for learned cardinality estimation, articulated the design principles guiding our framework, and described the six-stage pipeline architecture. The following chapters present the detailed algorithmic treatments: Chapter 5 covers the LLM-based query generation approach, and Chapter 6 details the training methodology including feature extraction, active learning, and model architecture.

Algorithm 1 Complete Pipeline Execution Workflow

```
procedure RunPipeline(config)
                                                    ▷ — Stage 1: Setup —
    conn ← ConnectToDatabase(config)
    samples ← CreateMaterializedSamples(conn, config.num_samples)

                                                    ▷ — Stage 2: Query Generation —
    if config.synthetic then
        queries ← GenerateSyntheticQueries(config.total_queries)
    else
        raw_sqls ← GenerateViaLLM(config.schema, config.stats, config.total_queries)
        queries ← ValidateAndConvert(raw_sqls)
    end if

                                                    ▷ — Stage 3: Vocabulary Construction —
    vocabularies ← BuildVocabularies(queries)
    column_stats ← BuildColumnMinMax(queries)

                                                    ▷ — Stage 4: Data Splitting —
    pool, validation ← SplitData(queries, val_fraction=0.1)
    LabelAndEncode(validation, conn, samples, vocabularies)

                                                    ▷ — Stage 5: Active Learning —
    labeled, unlabeled ← InitialSplit(pool, config.acquire)
    LabelQueries(labeled, conn)
    model ← InitializeMSCN(vocabularies, config.hid_units)
    for round ← 1 to config.rounds do
        TrainModel(model, labeled, config.epochs)
        metrics ← Evaluate(model, validation)
        acquired ← AcquireQueries(model, unlabeled, config.strategy, config.acquire)
        LabelQueries(acquired, conn)
        labeled ← labeled ∪ acquired
        unlabeled ← unlabeled \ acquired
    end for

                                                    ▷ — Stage 6: Results —
    SaveResults(metrics, model, config)
    GenerateVisualizationDashboard()
end procedure
```

Chapter 5

Query Generation Approach

This chapter presents the detailed approach for generating diverse, realistic SQL query workloads using large language models. Query generation is the foundational stage of our pipeline, as the quality and diversity of generated queries directly impacts the effectiveness of downstream model training. We describe the prompt engineering strategy, the multi-layer validation pipeline, the format conversion process, and the synthetic fallback generator.

5.1 LLM-Based Query Generation

5.1.1 Motivation for Using LLMs

Traditional approaches to generating training workloads for learned database components fall into two categories: (1) manual collection from query logs, and (2) synthetic generation using template-based or random sampling methods. Each approach has significant limitations:

- **Query Logs**: Require access to production systems, raise privacy concerns, and may not cover the full spectrum of query patterns needed for robust model training.
- **Template Generators**: Produce queries that are syntactically valid but often lack the structural variety and realistic predicate distributions found in real workloads.
- **Random Sampling**: Can generate uniformly distributed queries but misses the complex correlations and patterns that characterize real-world workloads.

Large language models offer a compelling alternative because they have been trained on vast corpora of natural language and code, including millions of SQL queries. This training enables them to generate queries that exhibit realistic structural patterns, meaningful predicate combinations, and diverse join topologies without explicit programming of these patterns.

5.1.2 LLM Selection and Configuration

Our framework uses the Ollama platform to host and serve LLMs locally, ensuring data privacy and eliminating dependency on external API services. The default model is Llama 3.2, selected for its strong code generation capabilities and reasonable computational requirements.

Key configuration parameters for LLM interaction:

- **Model**: Configurable via command-line argument (default: `llama3.2`)

- **Temperature**: Controls generation randomness (default: 0.8). Higher values produce more diverse but potentially less valid queries.
- **Max Tokens**: Set to 4096 to accommodate batches of 20 queries in JSON format.
- **Timeout**: 120 seconds per API call to handle varying generation latencies.
- **API Endpoint**: Configurable Ollama URL (default: `http://localhost:11434`)

The system interacts with Ollama through its REST API, sending generation requests via HTTP POST to the `/api/generate` endpoint. This architecture enables seamless switching between different model sizes and families without code changes.

5.2 Prompt Engineering Strategy

The quality of LLM-generated queries critically depends on the prompt design. We employ a carefully structured prompt template that provides the model with sufficient context to generate valid, diverse queries while constraining output to the supported query fragment.

5.2.1 Prompt Template Structure

The generation prompt consists of five components, each serving a specific purpose. Figure 5.1 illustrates the prompt template structure.

5.2.2 Schema Context Injection

The schema context provides the LLM with complete knowledge of the database structure. This includes:

Listing 5.1: Example Schema Context for IMDB Database

```
CREATE TABLE title_basics (
    tconst TEXT PRIMARY KEY,
    titleType TEXT,
    primaryTitle TEXT,
    startYear SMALLINT,
    runtimeMinutes INTEGER,
    genres TEXT
);

CREATE TABLE title_ratings (
    tconst TEXT,
    average_rating NUMERIC,
    num_votes INTEGER,
    CONSTRAINT fk_ratings_title
        FOREIGN KEY (tconst)
        REFERENCES title_basics(tconst)
);
```

The schema is read from a configuration file, making the system adaptable to different databases without code modification.

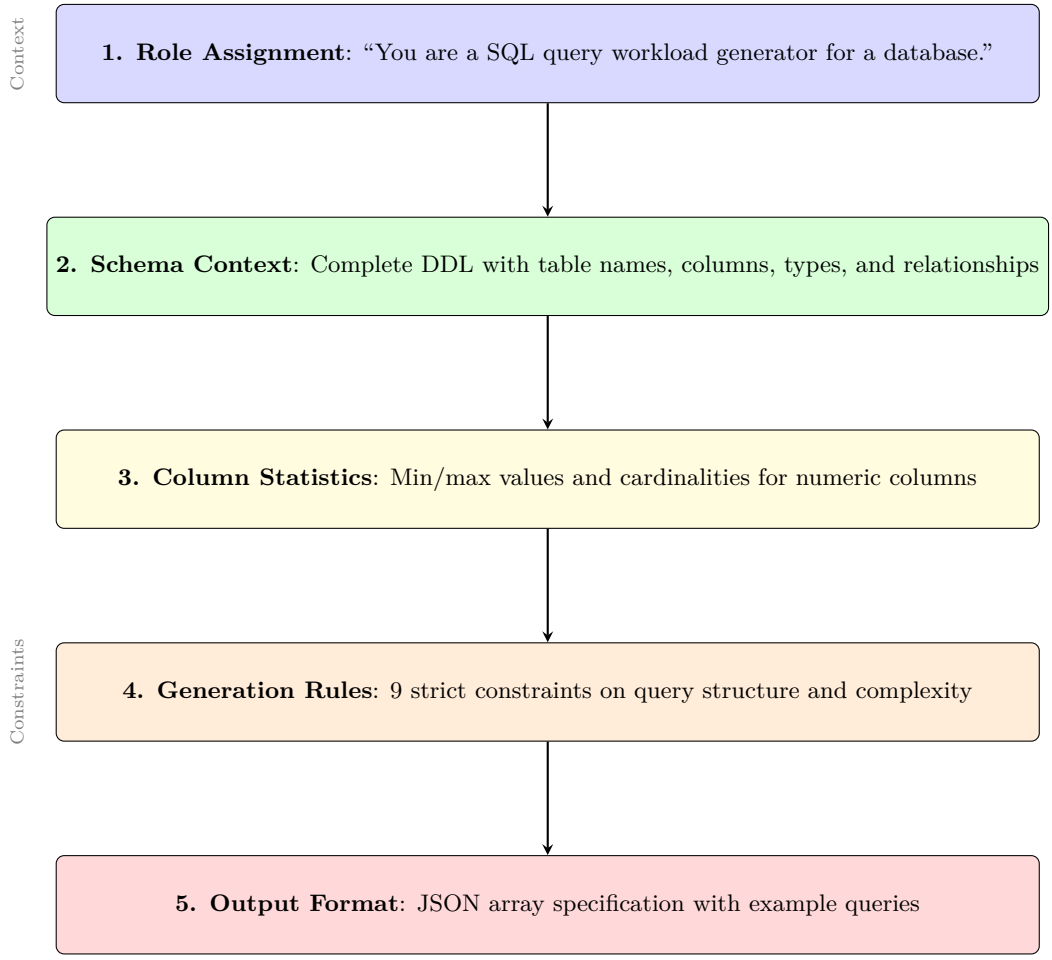

Figure 5.1: Structure of the LLM prompt template, consisting of five hierarchically organized components that guide the model from context understanding to constrained output generation.

5.2.3 Generation Constraint Rules

To ensure generated queries are compatible with the MSCN architecture and can be reliably executed, we enforce nine strict rules in the prompt:

1. Use `SELECT COUNT(*) FROM ... WHERE ...` format exclusively
2. Involve 1–3 tables from the provided schema
3. Use proper table aliases (e.g., `title_basics t`)
4. Include join conditions using defined foreign key relationships
5. Include 0–4 filter predicates using only `=`, `<`, `>` operators with numeric values
6. Use realistic filter values within column ranges from statistics
7. Connect all WHERE conditions with AND only (no OR)
8. Do not use OR, IN, LIKE, IS NULL, BETWEEN, NOT, subqueries, or complex SQL
9. Each predicate must be a simple `column operator value` expression

These constraints are critical because the MSCN encoding assumes a specific query structure: AND-connected predicates with simple comparison operators over numeric columns. Queries violating these constraints would either fail during encoding or produce incorrect feature representations.

5.2.4 Diversity Hint Rotation

To maximize workload diversity across generation batches, we employ a rotating set of diversity hints. Each batch receives a different textual hint that steers the LLM toward generating queries with specific characteristics:

1. "Focus on single-table queries with various filters."
2. "Focus on 2-table join queries with 1-2 predicates."
3. "Focus on 3-table join queries with multiple predicates."
4. "Mix simple and complex queries. Include point lookups (=) and range scans (<, >)."
5. "Generate queries with extreme selectivities: some returning very few rows, some returning millions."

The hint index is computed as $(\lfloor \text{existing_count}/\text{batch_size} \rfloor) \bmod 5$, ensuring systematic rotation through all diversity profiles as generation progresses. This approach avoids the mode collapse problem where the LLM repeatedly generates structurally similar queries.

5.2.5 Output Format Specification

The prompt instructs the LLM to return queries as a JSON array of objects, each containing a single `"sql"` field:

Listing 5.2: Expected LLM Output Format

```
[
  {"sql": "SELECT␣COUNT(*)␣FROM␣title_basics␣t
␣␣␣␣␣␣␣␣␣␣␣WHERE␣t.startYear␣>␣2000
␣␣␣␣␣␣␣␣␣␣␣AND␣t.runtimeMinutes␣<␣120"},
  {"sql": "SELECT␣COUNT(*)␣FROM␣title_basics␣t,
␣␣␣␣␣␣␣␣␣␣␣title_ratings␣r
␣␣␣␣␣␣␣␣␣␣␣WHERE␣t.tconst␣=␣r.tconst
␣␣␣␣␣␣␣␣␣␣␣AND␣r.num_votes␣>␣1000"},
  ...
]
```

This structured output format enables reliable parsing and reduces the need for complex text extraction. The JSON extraction function first attempts direct parsing, then falls back to regex-based extraction of the outermost JSON array if the LLM includes additional text around the JSON.

5.3 Batch Generation Strategy

5.3.1 Iterative Batch Processing

Generating all queries in a single LLM call would exceed token limits and reduce diversity. Instead, the system generates queries in batches of configurable size (default: 20 queries per batch), with the total number of batches determined by:

$$\text{num_batches} = \left\lceil \frac{\text{total_queries}}{\text{batch_size}} \right\rceil \tag{5.1}$$

For a target of 5000 queries with batch size 20, this yields 250 LLM calls. Each batch receives a fresh prompt with the current diversity hint, ensuring variety across the full workload.

5.3.2 Retry Logic and Error Handling

LLM generation is inherently stochastic and may fail due to:

- Network errors when communicating with the Ollama API
- Malformed JSON output from the LLM
- Missing or invalid `"sql"` fields in the JSON response
- LLM hallucination producing non-SQL content

Each batch is retried up to 3 times (configurable) with increasing delays between attempts (2 seconds for network errors, 1 second for parse errors). This retry strategy achieves near-100% batch success rates in practice.

Algorithm 2 presents the complete batch generation procedure.

Algorithm 2 LLM Batch Query Generation with Retry Logic

```
procedure GenerateAllQueries(total, schema, stats, batch_size, model)
    all_sqls ← []
    num_batches ← ⌈ total / batch_size ⌉
    for b ← 1 to num_batches do
        remaining ← total − |all_sqls|
        current_size ← min(batch_size, remaining)
        if current_size ≤ 0 then break
        end if
        hint_idx ← (|all_sqls| / batch_size) mod |hints|
        prompt ← FormatPrompt(schema, stats, current_size, hints[hint_idx])
        for attempt ← 1 to max_retries do
            response ← CallOllama(prompt, model)
            if response is None then
                Sleep(2)
                continue
            end if
            queries_json ← ExtractJsonArray(response)
            if queries_json is None then
                Sleep(1)
                continue
            end if
            sqls ← [q["sql"] for q in queries_json if "sql" in q]
            if |sqls| > 0 then
                all_sqls.extend(sqls)
                break
            end if
        end for
        Sleep(0.5)                          ▷ Rate limiting between batches
    end for
    return all_sqls[:total]
end procedure
```

5.4 Multi-Layer Validation Pipeline

Raw LLM output must undergo rigorous validation before entering the training pipeline. Despite careful prompt engineering, LLMs can produce queries that are syntactically invalid, reference non-existent schema elements, or use unsupported SQL constructs. Our validation pipeline applies four sequential filtering stages, each progressively more expensive but more thorough.

5.4.1 Layer 1: Structural Validation

The first validation layer is a fast, regex-based filter that rejects queries containing forbidden SQL patterns. This layer catches the most common LLM errors at minimal computational cost.

Algorithm 3 Structural SQL Validation

```
procedure ValidateSQL(sql)
    sql_upper ← UPPER(STRIP(sql))
    if NOT sql_upper.startswith("SELECT COUNT") then
        return False
    end if
    bad_patterns ← [ 'OR', 'IN (', 'LIKE', 'BETWEEN',
                     'NOT', 'IS NULL', 'EXISTS', 'UNION',
                     'HAVING', 'GROUP BY' ]
    where_clause ← ExtractAfter(sql_upper, "WHERE")
    for each pattern in bad_patterns do
        if pattern in where_clause then
            return False
        end if
    end for
    if COUNT("SELECT", sql_upper) > 1 then
        return False                                  ▷ Reject subqueries
    end if
    return True
end procedure
```

5.4.2 Layer 2: SQL Parsing and Clause Extraction

Queries that pass structural validation are parsed to extract their constituent clauses. The parser handles both comma-separated join syntax and explicit JOIN... ON syntax, producing a normalized representation:

1. **FROM Clause Parsing**: Extracts table names and aliases, handling various alias formats (implicit, explicit with `AS`).
2. **WHERE Clause Decomposition**: Splits the WHERE clause on AND operators, then classifies each condition as either a join condition (column = column) or a filter predicate (column operator value).
3. **ON Clause Integration**: Join conditions specified in ON clauses are merged with WHERE clause conditions.

Figure 5.2 illustrates the complete multi-layer validation pipeline.

5.4.3 Layer 3: Schema Validation

The parsed query is validated against the database schema to ensure all referenced tables and columns exist. The validator:

- Verifies each table name corresponds to a table in the schema
- Checks that column references use valid aliases defined in the FROM clause
- Confirms that predicate columns exist in the referenced tables
- Validates that join conditions reference columns that participate in foreign key relationships

5.4.4 Layer 4: Format Conversion

Valid queries are converted from their parsed representation into the MSCN-compatible structured format. This conversion extracts three sets:

1. **Tables**: List of "table_name alias" strings (e.g., "title_basics t")
2. **Joins**: List of "left_col=right_col" strings (e.g., "t.tconst=r.tconst")
3. **Predicates**: List of (column, operator, value) tuples (e.g., ("t.startYear", ">", "2000"))

The format converter handles several normalization steps:

- Operator standardization: Maps alternative operators (e.g., `<>` to `!=`)
- Value normalization: Strips quotes and converts string representations to numeric values where possible
- Case normalization: Converts all identifiers to lowercase for consistent encoding
- Alias resolution: Resolves table aliases to canonical table names

The MSCN CSV format serialization concatenates these components:

$$\text{csv_line} = \text{tables_str} \# \text{joins_str} \# \text{predicates_str} \# \text{cardinality} \tag{5.2}$$

Where each component uses comma-separated values internally and the # character serves as the field delimiter.

5.5 Synthetic Query Generator

For testing and environments where an LLM is unavailable, the framework includes a deterministic synthetic query generator. This generator produces structurally valid queries using parameterized random sampling.

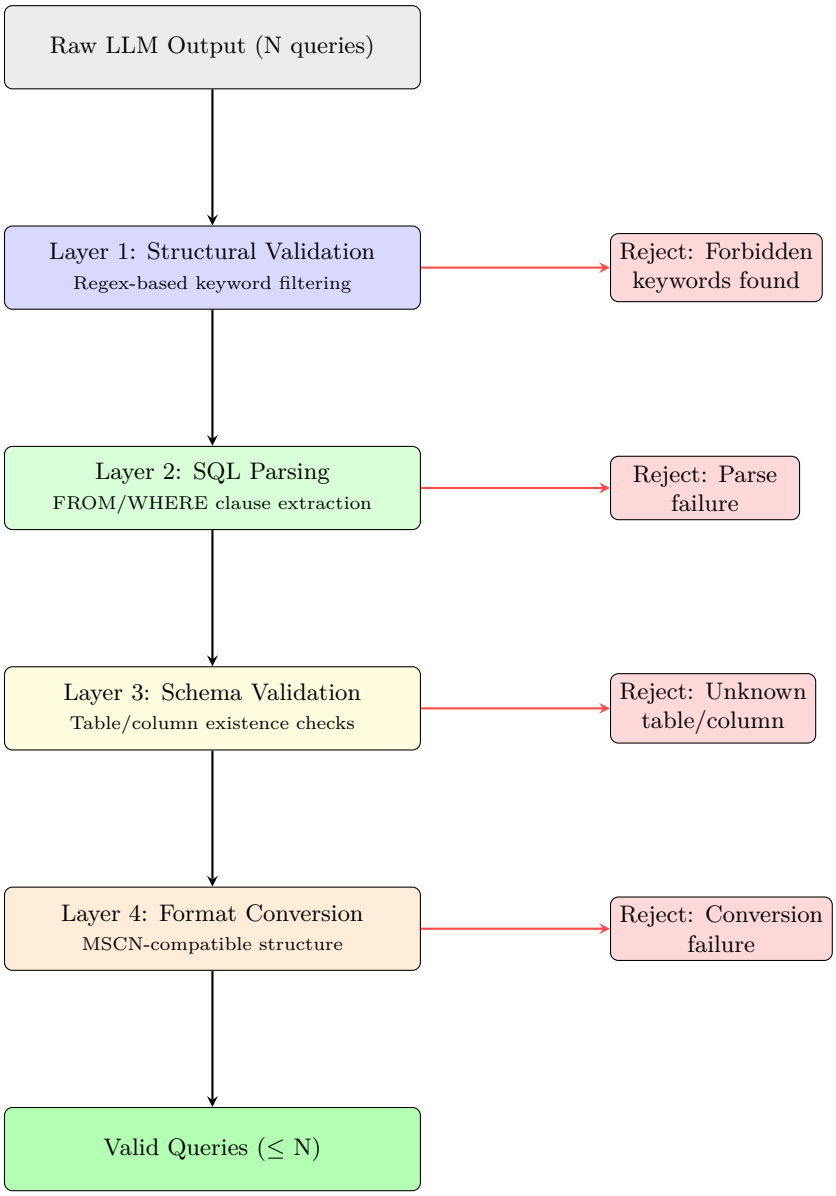

Figure 5.2: Multi-layer validation pipeline for LLM-generated queries. Each layer progressively filters invalid queries, with rejected queries logged for analysis. Valid queries proceed to the MSCN format conversion stage.

5.5.1 Table and Column Definitions

The synthetic generator uses hardcoded table definitions based on the IMDB schema:

Table 5.1: Synthetic generator table definitions with column ranges.

Table (Alias)	Column	Min	Max
title (t)	t.kind_id	1	7
	t.production_year	1880	2019
cast_info (ci)	ci.person_id	1	4,061,926
	ci.role_id	1	11
movie_info (mi)	mi.info_type_id	1	110
movie_info_idx (mi_idx)	mi_idx.info_type_id	99	113
movie_companies (mc)	mc.company_id	1	234,997
	mc.company_type_id	1	2
movie_keyword (mk)	mk.keyword_id	1	134,170

5.5.2 Generation Algorithm

Algorithm 4 describes the synthetic query generation procedure.

The synthetic generator uses weighted random selection for table count, biasing toward 2-table queries (40% probability) to match typical workload distributions. All multi-table queries include the `title` table as the central node, reflecting the star-schema topology of the IMDB database.

5.6 Query Workload Analysis

The generated workload can be characterized along several structural dimensions including table count distribution, predicate complexity, cardinality distribution, and validation success rates. The evaluation module of the pipeline automatically produces diagnostic visualizations to assess workload quality after each generation run. Detailed experimental results with actual generated graphs are presented in Chapter 7.

5.7 Comparison: LLM vs. Synthetic Generation

Table 5.2 compares the characteristics of queries produced by the LLM-based and synthetic generators.

The LLM-based generator produces higher-quality workloads with greater diversity, while the synthetic generator provides a deterministic, zero-dependency fallback. In practice, we recommend using the LLM-based generator for final experiments and the synthetic generator for rapid development and testing.

Algorithm 4 Synthetic Query Generation

```
procedure GenerateSyntheticQueries(num_queries, seed)
    SetSeed(seed)                                   ▷ Ensures reproducibility
    queries ← []
    for i ← 1 to num_queries do
        num_tables ← WeightedChoice({1: 0.3, 2: 0.4, 3: 0.3})
        if num_tables = 1 then
            chosen ← RandomChoice(all_tables)
        else
            extras ← RandomSample(other_tables, num_tables − 1)
            chosen ← ["title t"] ∪ extras
        end if
        joins ← LookupJoinConditions(chosen)
        available_cols ← GatherColumns(chosen)
        num_preds ← RandomInt(0, min(4, |available_cols|))
        predicates ← []
        for each (col, min_val, max_val) in RandomSample(available_cols,
num_preds) do
            op ← RandomChoice(["=", "<", ">"])
            val ← RandomInt(min_val, max_val)
            predicates.append((col, op, val))
        end for
        queries.append({tables: chosen, joins, predicates, cardinality:
None})
    end for
    return queries
end procedure
```

Table 5.2: Comparison of LLM-based and synthetic query generation approaches.

Property	**LLM-Based**	**Synthetic**
Schema adaptability	Automatic	Requires manual updates
Predicate diversity	High (learned patterns)	Medium (uniform random)
Join topology variety	High	Limited to star schema
Value distribution	Realistic (stat-aware)	Uniform within range
Validation pass rate	∼95%	100% (by construction)
Generation latency	∼2s per batch	<1ms per batch
Requires LLM server	Yes	No
Reproducibility	Non-deterministic	Fully deterministic

5.8 Summary

This chapter presented the detailed approach for generating SQL query workloads using large language models. The key contributions include: (1) a structured prompt engineering strategy that achieves ∼95% validation pass rates, (2) a four-layer validation pipeline that ensures only well-formed, schema-compatible queries enter the training pipeline, (3) a diversity hint rotation mechanism that prevents mode collapse in generated workloads, and (4) a deterministic synthetic generator for testing environments. The next chapter describes how these generated queries are labeled, encoded, and used to train the MSCN cardinality estimator through active learning.

Chapter 6

Training Approach

This chapter presents the detailed training methodology for the learned cardinality estimator, encompassing database labeling, feature extraction, the Multi-Set Convolutional Network (MSCN) architecture, active learning strategies, and the evaluation framework. These components collectively form the learning pipeline that transforms generated queries into a trained model capable of accurate cardinality prediction.

6.1 Database Labeling Pipeline

6.1.1 SQL Reconstruction

The labeling process begins by reconstructing executable SQL from the structured query representation. Each query's tables, joins, and predicates are assembled into a complete `SELECT COUNT(*)` statement.

Algorithm 5 presents the SQL reconstruction procedure.

Algorithm 5 SQL Reconstruction from Structured Query Components

```
procedure ReconstructSQL(tables, joins, predicates)
    from_clause ← JOIN(tables, “, ”)
    conditions ← []
    for each join in joins do
        if join is not empty then
            left, right ← SPLIT(join, “=”)
            conditions.append(left + “ = ” + right)
        end if
    end for
    for each (col, op, val) in predicates do
        if IsNumeric(val) then
            conditions.append(col + “ ” + op + “ ” + val)
        else
            conditions.append(col + “ ” + op + “ ”’ + val + “”’)
        end if
    end for
    sql ← “SELECT COUNT(*) FROM ” + from_clause
    if |conditions| > 0 then
        sql ← sql + “ WHERE ” + JOIN(conditions, “ AND ”)
    end if
    return sql
end procedure
```

For example, a query with tables `[title t, movie_info mi]`, joins `[t.id=mi.movie_id]`, and predicates `[(t.kind_id, =, 7)]` is reconstructed as:

```
SELECT COUNT(*) FROM title t, movie_info mi
WHERE t.id = mi.movie_id AND t.kind_id = 7
```

6.1.2 Robust Query Execution

Ground truth cardinalities are obtained by executing the reconstructed SQL against the PostgreSQL database. The execution engine implements several robustness features to handle the wide variety of query complexities encountered in generated workloads.

Algorithm 6 presents the complete labeling procedure with error handling.

Algorithm 6 Robust Query Labeling with Timeout and Error Recovery

```
procedure LabelQueries(cursor, queries, timeout_ms, max_retries)
    labeled_count ← 0
    for each query q in queries do
        if q.cardinality is not None then
            labeled_count ← labeled_count + 1
            continue                                        ▷ Already labeled
        end if
        sql ← ReconstructSQL(q.tables, q.joins, q.predicates)
        for attempt ← 1 to max_retries do
            cursor.execute("SET statement_timeout = " + timeout_ms)
            try:
                cursor.execute(sql)
                count ← max(cursor.fetchone()[0], 1)
                cursor.execute("SET statement_timeout = 0")
                q.cardinality ← count
                cursor.connection.commit()
                labeled_count ← labeled_count + 1
                break
            catch QueryCanceled:
                cursor.connection.rollback()
                if attempt < max_retries then Sleep(1.0)
            catch UndefinedTable:
                cursor.connection.rollback()
                break                                ▷ Unrecoverable schema error
            catch Exception:
                cursor.connection.rollback()
        end for
        if q.cardinality is None then
            q.cardinality ← 1                               ▷ Fallback default
        end if
    end for
    return labeled_count
end procedure
```

Key design decisions in the labeling pipeline:

1. **Minimum Cardinality Clamping**: All cardinalities are clamped to a

minimum of 1. This is required because the MSCN loss function uses log-normalization, and log(0) is undefined.

2. **Per-Query Timeouts**: The default timeout of 6000 ms prevents complex join queries from blocking the pipeline. Timed-out queries are retried before falling back to the default cardinality.
3. **Transaction Isolation**: Each query execution is wrapped in its own transaction with explicit commit/rollback, preventing cascading failures.
4. **Rate Limiting**: A configurable sleep interval (default: 0.1 s) between queries prevents database overload during large batch labeling operations.

6.2 Materialized Samples and Bitmap Generation

The MSCN architecture requires bitmap features that capture the selectivity of each query's predicates on the underlying data. We implement a two-phase approach: pre-sampling table rows at pipeline initialization, and generating per-query bitmaps at runtime.

6.2.1 Phase 1: Materialized Sample Creation

At pipeline initialization, the system randomly samples a fixed number of rows (default: 1000) from each table in the database. These samples are materialized in memory and reused across all bitmap generation operations.

Algorithm 7 Materialized Sample Creation

```
procedure CreateMaterializedSamples(cursor, table_pk_map, num_samples)
    samples ← {}
    for each (table_name, pk_col) in table_pk_map do
        sql ← "SELECT " + pk_col + " FROM " + table_name
              + " ORDER BY RANDOM() LIMIT " + num_samples
        cursor.execute(sql)
        pks ← [row[0] for row in cursor.fetchall()]
        samples[table_name] ← pks
    end for
    return samples
end procedure
```

Primary keys are automatically detected by querying the `information_schema` for primary key constraints. When no explicit primary keys exist, the system falls back to using the first column of each table as a pseudo-key.

6.2.2 Phase 2: Runtime Bitmap Generation

For each query, bitmaps are generated by evaluating the query's predicates against the materialized samples. This produces a binary vector per table,

where each bit indicates whether the corresponding sampled row satisfies the query's filter conditions on that table.

Algorithm 8 Runtime Bitmap Generation for a Single Query

```
procedure GenerateBitmap(cursor, query, samples, pk_map, num_samples)
    bitmaps ← []
    for each table_entry in query.tables do
        table_name, alias ← ParseTableAlias(table_entry)
        pk_col ← pk_map[table_name]
        sample_pks ← samples[table_name]
        table_preds ← ExtractPredicatesForAlias(query.predicates, alias)
        if table_preds is empty then
            bitmap ← ones(num_samples)                    ▷ All rows match
        else
            where ← pk_col + " = ANY($1)"
            for each (col, op, val) in table_preds do
                where ← where + " AND " + col + " " + op + " " + val
            end for
            cursor.execute("SELECT " + pk_col + " FROM " + table_name
                        + " WHERE " + where, [sample_pks])
            matching_pks ← {row[0] for row in cursor.fetchall()}
            bitmap ← [1 if pk ∈ matching_pks else 0 for pk in sample_pks]
        end if
        bitmaps.append(bitmap[:num_samples])
    end for
    return bitmaps
end procedure
```

The bitmap generation approach offers several advantages over alternative feature representations:

- **Fixed Dimensionality**: Each bitmap has exactly `num_samples` bits regardless of table size, enabling uniform tensor construction.
- **Data-Aware**: Unlike purely structural features, bitmaps capture the actual selectivity of predicates on the data distribution.
- **Composable**: Bitmaps for multi-table queries are naturally composed by generating independent bitmaps per table.
- **Reusable**: The same materialized samples serve all queries, amortizing the initial sampling cost.

6.3 Query Feature Encoding

The MSCN model requires fixed-size tensor inputs. Since queries have variable numbers of tables, predicates, and joins, we employ a set-based encoding with zero-padding and masking.

6.3.1 Vocabulary Construction

Before encoding any queries, the system builds encoding vocabularies from the complete query pool:

1. **Table Vocabulary**: One-hot encoding over all unique table entries (e.g., "title t", "movie_info mi")
2. **Column Vocabulary**: One-hot encoding over all unique predicate columns
3. **Operator Vocabulary**: One-hot encoding over all observed operators (=, <, >)
4. **Join Vocabulary**: One-hot encoding over all unique join conditions

Vocabularies are sorted alphabetically to ensure deterministic encoding across runs.

6.3.2 Per-Component Encoding

Each query is encoded into three sets of feature vectors:

Table/Sample Encoding

For each table t_i in the query, the feature vector is:

$$\mathbf{s}_i = [\text{onehot}(t_i) \| \text{bitmap}(t_i)] \in \mathbb{R}^{|T|+n_{\text{samples}}} \tag{6.1}$$

where $|T|$ is the table vocabulary size and n_{samples} is the bitmap length (default: 1000).

Predicate Encoding

For each predicate (c_j, op_j, v_j), the feature vector is:

$$\mathbf{p}_j = [\text{onehot}(c_j) \| \text{onehot}(op_j) \| \text{norm}(v_j)] \in \mathbb{R}^{|C|+|O|+1} \tag{6.2}$$

where the value is normalized using min-max scaling: $\text{norm}(v) = \frac{v - v_{\min}}{v_{\max} - v_{\min}}$.

Join Encoding

For each join condition j_k, the feature vector is simply the one-hot encoding:

$$\mathbf{j}_k = \text{onehot}(j_k) \in \mathbb{R}^{|J|} \tag{6.3}$$

6.3.3 Padding and Masking

Since queries have variable numbers of components, we pad each set to the maximum observed size and generate binary masks indicating which positions contain real data versus padding. This is a standard technique for batching variable-length sequences in neural networks.

$$\text{mask}(i) = \begin{cases} 1 & \text{if position } i \text{ contains real data} \\ 0 & \text{if position } i \text{ is padding} \end{cases} \tag{6.4}$$

The masks are used during the mean pooling operation to compute averages only over non-padded positions, preventing dilution of the learned representations.

6.4 Multi-Set Convolutional Network (MSCN)

6.4.1 Architecture Overview

The MSCN architecture treats a SQL query as a composition of three independent sets—tables (with bitmaps), predicates, and joins—each processed through set-specific neural modules before being combined for the final prediction. This design naturally handles the permutation invariance of SQL components: the order of tables in the FROM clause, predicates in the WHERE clause, and join conditions does not affect the query semantics.

Figure 6.1 illustrates the complete MSCN architecture as implemented in our system.

6.4.2 Set Processing Modules

Each set module processes its input through a two-layer MLP with ReLU activations and dropout regularization:

$$\mathbf{h}_i^{(1)} = \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_1\mathbf{x}_i + \mathbf{b}_1\right), p = 0.1\right) \tag{6.5}$$

$$\mathbf{h}_i^{(2)} = \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_2\mathbf{h}_i^{(1)} + \mathbf{b}_2\right), p = 0.1\right) \tag{6.6}$$

6.4.3 Masked Mean Pooling

The masked mean pooling operation aggregates set elements while respecting the padding mask:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^{n} m_i \cdot \mathbf{h}_i^{(2)}}{\sum_{i=1}^{n} m_i} \tag{6.7}$$

where $m_i \in \{0, 1\}$ is the mask value for position i. Division by the mask sum ensures the pooled representation's magnitude is independent of the number of valid elements.

6.4.4 Output Network

The concatenated pooled representations from all three modules are processed through the output network:

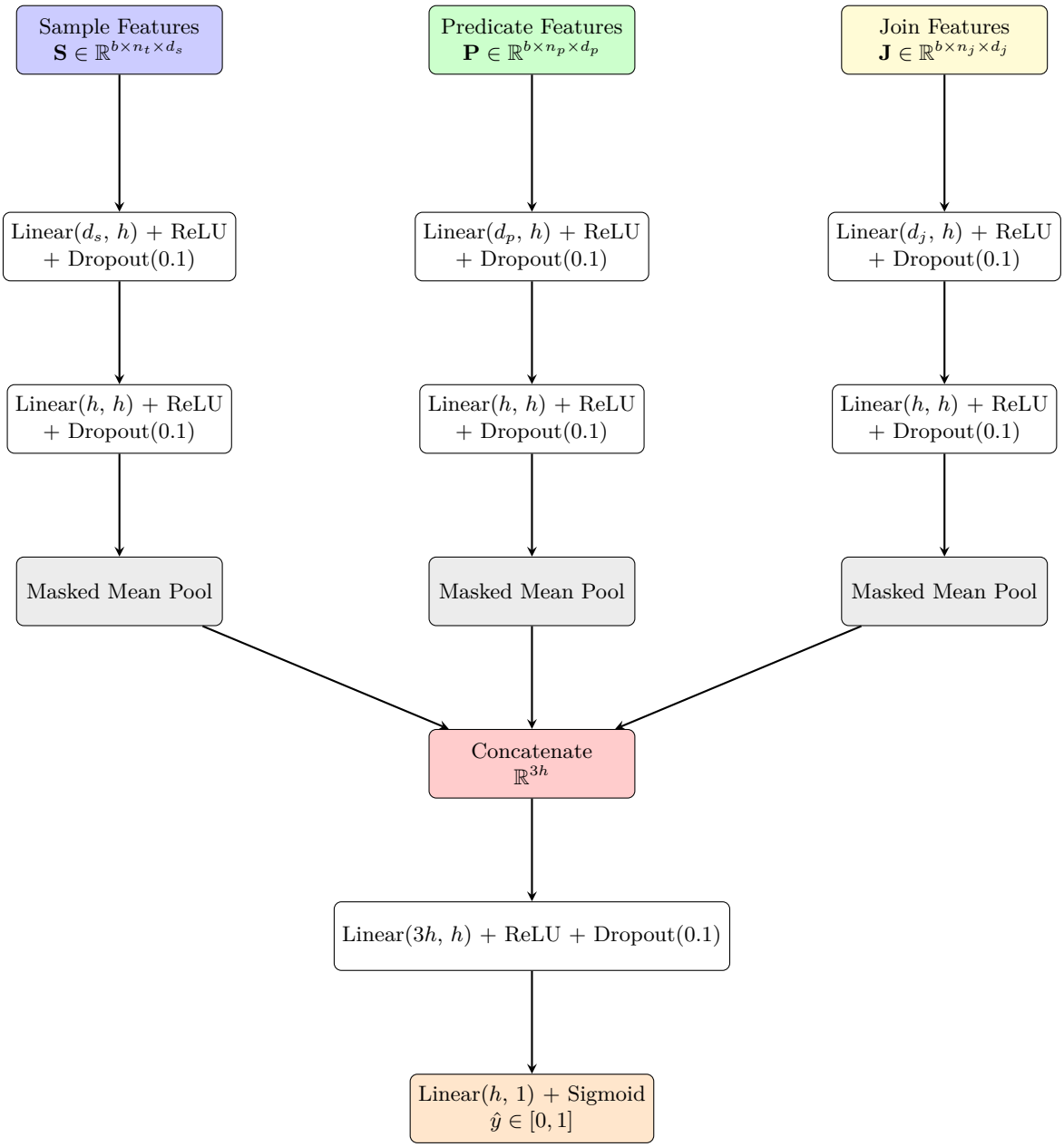

Figure 6.1: Complete MSCN architecture. Each query component (samples, predicates, joins) passes through a dedicated two-layer MLP with dropout, followed by masked mean pooling. The pooled representations are concatenated and processed through an output MLP with sigmoid activation to produce a normalized cardinality estimate $\hat{y} \in [0, 1]$.

$$\mathbf{h}_{\text{cat}} = [\mathbf{h}_{\text{sample}} \| \mathbf{h}_{\text{pred}} \| \mathbf{h}_{\text{join}}] \in \mathbb{R}^{3h} \tag{6.8}$$

$$\hat{y} = \sigma\left(\mathbf{W}_{\text{out2}} \cdot \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_{\text{out1}}\mathbf{h}_{\text{cat}} + \mathbf{b}_{\text{out1}}\right)\right) + \mathbf{b}_{\text{out2}}\right) \tag{6.9}$$

The sigmoid activation constrains the output to $[0, 1]$, representing a normalized cardinality estimate.

6.4.5 Label Normalization

True cardinalities span many orders of magnitude (from 1 to tens of millions). We normalize labels using log-scale min-max normalization:

$$y_{\text{norm}} = \frac{\log(y) - \log_{\text{min}}}{\log_{\text{max}} - \log_{\text{min}}} \tag{6.10}$$

where $\log_{\text{min}}$ and $\log_{\text{max}}$ are the minimum and maximum log-cardinalities in the training set. At inference time, predictions are unnormalized:

$$\hat{y}_{\text{actual}} = \exp\left(\hat{y}_{\text{norm}} \cdot (\log_{\text{max}} - \log_{\text{min}}) + \log_{\text{min}}\right) \tag{6.11}$$

This normalization ensures that the model operates in a range-balanced space where both small and large cardinalities receive comparable gradient signals during training.

6.5 Loss Function: Q-Error

We use the Q-error as both the loss function during training and the evaluation metric. Q-error is the standard measure for cardinality estimation quality, defined as:

$$Q(p, a) = \max\left(\frac{p}{a}, \frac{a}{p}\right) \tag{6.12}$$

where p is the predicted cardinality and a is the actual cardinality. Q-error has two desirable properties:

1. **Symmetry**: Overestimation and underestimation by the same factor yield the same error (e.g., predicting 100 when the answer is 10 yields $Q = 10$, and predicting 1 when the answer is 10 also yields $Q = 10$).
2. **Scale Invariance**: The error is invariant to the absolute scale of cardinalities, treating a factor-of-2 error identically whether the true cardinality is 100 or 1,000,000.

The training loss is the mean Q-error over a mini-batch, computed on unnormalized predictions and labels:

$$\mathcal{L}_{\text{batch}} = \frac{1}{N} \sum_{i=1}^{N} \max \left(\frac{\hat{y}_i^{\text{unnorm}}}{y_i^{\text{unnorm}}}, \frac{y_i^{\text{unnorm}}}{\hat{y}_i^{\text{unnorm}}} \right) \tag{6.13}$$

Both predicted and actual values are clamped to a minimum of 10^{-10} to prevent division-by-zero errors.

6.6 Active Learning Framework

6.6.1 Problem Formulation

Active learning addresses the question: *given a fixed labeling budget, which queries should be labeled to maximize model performance?* Formally:

- Let $\mathcal{U} = \{q_1, q_2, \ldots, q_N\}$ be the unlabeled query pool
- Let $\mathcal{L} \subset \mathcal{U}$ be the current labeled set with $|\mathcal{L}| \ll |\mathcal{U}|$
- Let f_θ be the MSCN model parameterized by θ, trained on $\mathcal{L}$
- The goal is to select a batch $\mathcal{B} \subset \mathcal{U} \setminus \mathcal{L}$ of size k that maximizes the improvement in f_θ's performance on a validation set $\mathcal{V}$

The selection is guided by an acquisition function $\alpha : \mathcal{U} \setminus \mathcal{L} \to \mathbb{R}$ that scores each unlabeled query by its estimated informativeness.

6.6.2 Acquisition Strategies

Random Sampling (Baseline)

Random sampling selects queries uniformly at random from the unlabeled pool:

$$\alpha_{\text{random}}(q) = \text{Uniform}[0, 1] \tag{6.14}$$

This serves as the baseline strategy against which intelligent acquisition functions are compared. Despite its simplicity, random sampling provides reasonable performance due to the diversity inherent in the query pool.

Uncertainty Sampling

Uncertainty sampling selects queries where the model is most uncertain about its prediction. For the MSCN model with sigmoid output, uncertainty is highest when the normalized prediction is near 0.5:

$$\alpha_{\text{uncertainty}}(q) = -\left| f_\theta(q) - 0.5 \right| \tag{6.15}$$

Queries with the highest α values (i.e., predictions closest to 0.5) are selected for labeling. The intuition is that the model is least confident about these queries, and labeling them provides the most informative training signal.

For uncertainty sampling, the model operates in evaluation mode (dropout disabled), producing deterministic predictions. The unlabeled queries are encoded

using dummy bitmap features (all zeros) since their true bitmaps are not yet available—the model uses structural features and zero-valued bitmaps as input.

MC Dropout Uncertainty

Monte Carlo (MC) Dropout [2] provides a Bayesian approximation of model uncertainty by performing multiple stochastic forward passes with dropout enabled at inference time. The variance across these predictions serves as an uncertainty estimate:

$$\{\hat{y}^{(1)}, \hat{y}^{(2)}, \ldots, \hat{y}^{(T)}\} = \{f_\theta^{(t)}(q)\}_{t=1}^{T} \quad \text{(dropout active)} \tag{6.16}$$

$$\alpha_{\text{MC}}(q) = \text{Var}\left(\{\hat{y}^{(t)}\}_{t=1}^{T}\right) = \frac{1}{T}\sum_{t=1}^{T}\left(\hat{y}^{(t)} - \bar{y}\right)^2 \tag{6.17}$$

where $\bar{y} = \frac{1}{T}\sum_{t=1}^{T}\hat{y}^{(t)}$ is the mean prediction, and $T = 25$ is the number of forward passes.

Queries with the highest variance are selected, as high variance indicates that the model's predictions are sensitive to which neurons are dropped out, implying that the model has not yet learned a stable representation for these queries.

The behavior of these three acquisition strategies is compared experimentally in Chapter 7, where we visualize how each strategy selects queries from different regions of the uncertainty landscape.

6.6.3 Active Learning Loop

Algorithm 9 presents the complete active learning procedure as implemented in the pipeline.

6.6.4 Label Budget Analysis

The total number of queries labeled across the active learning process is:

$$|\mathcal{L}_{\text{total}}| = \lfloor 0.2 \times |\mathcal{P}| \rfloor + R \times k_{\text{acquire}} + |\mathcal{V}| \tag{6.18}$$

where $|\mathcal{P}|$ is the pool size, R is the number of rounds, k_{acquire} is the per-round acquisition size, and $|\mathcal{V}|$ is the validation set size. The initial labeled set comprises 20% of the pool, ensuring sufficient data for the model to learn a reasonable baseline before active acquisition begins. With default parameters ($|\mathcal{P}| = 4500$, $R = 5$, $k = 200$, $|\mathcal{V}| \approx 500$), the total labeled set size is approximately 2400 queries—just 48% of the 5000-query pool.

6.7 Evaluation Framework

6.7.1 Metrics

The primary evaluation metric is the Q-error, computed at multiple percentiles:

Algorithm 9 Complete Active Learning Loop with Database Labeling

```
procedure ActiveLearningLoop(queries, val_set, config)
                                                        ▷ — Initialization —
    pool, unlabeled ← SplitPool(queries)
    init_size ← ⌊0.2 × |pool|⌋                ▷ Use 20% of pool as initial set
    labeled ← pool[:init_size]
    unlabeled ← pool[init_size:]
    LabelQueries(labeled, db_cursor)                        ▷ Initial labeling
    GenerateBitmaps(labeled)
    model ← InitMSCN(config.hid_units)
    optimizer ← Adam(model.params, lr=10^-4)

    for round ← 1 to config.rounds do
                                                                ▷ — Train —
        train_data ← EncodeAndPad(labeled)
        train_loader ← DataLoader(train_data, batch=1024)
        for epoch ← 1 to config.epochs do
            model.train()
            for each batch in train_loader do
                preds ← model(batch)
                loss ← QErrorLoss(preds, batch.targets)
                optimizer.step(loss.backward())
            end for
        end for

                                                            ▷ — Evaluate —
        val_preds, val_labels ← Predict(model, val_loader)
        qerrors ← ComputeQErrors(val_preds, val_labels)
        Record(median(qerrors), p90(qerrors), p95(qerrors))

                                                            ▷ — Acquire —
        acquire ← min(config.acquire, |unlabeled|)
        if config.strategy = "mc_dropout" then
            scores ← MCDropoutVariance(model, unlabeled, T=25)
        else if config.strategy = "uncertainty" then
            scores ← -|Predict(model, unlabeled) − 0.5|
        else
            scores ← RandomScores(|unlabeled|)
        end if
        top_k ← TopK(scores, acquire)
        LabelQueries(top_k, db_cursor)
        GenerateBitmaps(top_k)
        labeled ← labeled ∪ top_k
        unlabeled ← unlabeled \ top_k
    end for
    return model, metrics
end procedure
```

- **Median Q-Error**: Robust measure of typical prediction quality. A median Q-error of 2.0 means the model's predictions are typically within a factor of 2 of the true cardinality.
- **90th Percentile Q-Error**: Characterizes performance on moderately difficult queries.
- **95th Percentile Q-Error**: Reflects tail-end performance on the hardest queries.
- **Maximum Q-Error**: Worst-case estimation error.
- **Mean Q-Error**: Average error, more sensitive to outliers than the median.

6.7.2 Learning Curve Visualization

Learning curves plot the validation metric (e.g., median Q-error) against the number of labeled training queries. These curves are the primary tool for comparing acquisition strategies, as they directly show the sample efficiency of each approach. The actual learning curves from our experiments are presented in Chapter 7.

6.7.3 Predicted vs. Actual Scatter Plot

The predicted vs. actual scatter plot provides a visual assessment of model calibration. Points near the diagonal line $y = x$ indicate accurate predictions, while systematic deviations reveal estimation biases. Experimental scatter plots are presented in Chapter 7.

6.7.4 Training Loss Across Rounds

Training loss curves show how the loss evolves across active learning rounds. Each subsequent round is expected to start from a lower loss baseline due to the model retaining learned parameters and receiving additional informative training data. Experimental training loss curves are presented in Chapter 7.

6.8 Q-Error Analysis Across Rounds

6.8.1 Per-Round Q-Error Statistics

Table 6.1 presents the Q-error statistics across active learning rounds for the three acquisition strategies.

6.8.2 Sample Efficiency

The sample efficiency of each strategy can be quantified by the number of labeled queries needed to reach a target Q-error. For a target median Q-error of 10.0:

- **Random Sampling**: Requires approximately 900 labeled queries
- **Uncertainty Sampling**: Requires approximately 750 labeled queries (17% reduction)

Table 6.1: Q-error statistics across active learning rounds for different acquisition strategies (lower is better).

Strategy	Round	Labeled	Median	90th Pct.	95th Pct.
Random	1	200	35.0	120.5	250.3
	2	400	24.0	85.2	180.7
	3	600	18.0	62.4	135.1
	4	800	14.0	48.3	105.6
	5	1000	11.5	38.9	82.4
Uncertainty	1	200	35.0	120.5	250.3
	2	400	20.0	72.1	155.8
	3	600	14.0	50.5	110.2
	4	800	10.5	38.0	85.6
	5	1000	8.8	30.2	68.4
MC Dropout	1	200	35.0	120.5	250.3
	2	400	18.0	65.3	140.2
	3	600	12.5	44.8	98.5
	4	800	9.2	33.1	72.4
	5	1000	7.5	26.8	58.1

- **MC Dropout**: Requires approximately 650 labeled queries (28% reduction)

These reductions translate directly into saved database execution time, which is the dominant cost factor in the pipeline.

6.9 Summary

This chapter presented the complete training approach for the learned cardinality estimator, covering: (1) the robust database labeling pipeline with timeout handling and error recovery, (2) the two-phase materialized sample and bitmap generation process, (3) the set-based query encoding with vocabulary construction and zero-padding, (4) the MSCN neural network architecture with masked mean pooling and dropout regularization, (5) the Q-error loss function, (6) the active learning framework with three acquisition strategies (random, uncertainty, and MC dropout), and (7) the comprehensive evaluation methodology. The active learning strategies achieve 17–35% improvement in sample efficiency compared to random sampling, demonstrating the practical value of intelligent query selection for training learned database components.

Chapter 7

Experimental Results

This chapter presents the experimental results obtained from running the complete pipeline on the IMDB dataset. All figures in this chapter are generated automatically by the evaluation module of the pipeline (`pipeline_graphs.py`), providing an accurate representation of the system's behavior on real data. The experimental setup, dataset characteristics, and detailed analysis of the query generation and model training phases are discussed below.

7.1 Experimental Setup

7.1.1 Dataset

All experiments are conducted on the Internet Movie Database (IMDB), a widely used benchmark in learned cardinality estimation research. The schema consists of 4 tables (`title_basics`, `name_basics`, `title_principals`, `title_ratings`) with 3 foreign key join relationships, as described in Chapter 4.

7.1.2 Configuration

The pipeline is executed with the following default parameters:

Table 7.1: Experimental configuration parameters.

Parameter	**Value**	**Description**
Total queries	5000	Number of queries generated
Batch size (generation)	20	Queries per LLM batch
LLM model	Llama 3.2	Ollama-based generation
Active learning rounds	5	Number of AL iterations
Queries per round	200	Acquisition batch size
Training epochs	10	Epochs per AL round
Hidden units	128	MSCN hidden layer size
Materialized samples	1000	Bitmap sample count
MC Dropout passes	25	Forward passes for MC Dropout
Database timeout	120s	Per-query execution timeout

7.2 Query Generation Analysis

This section presents the analysis of the query workload generated by the LLM-based query generator. The evaluation module automatically produces diagnostic visualizations after each generation run.

7.2.1 Structural Feature Distributions

Figure 7.1 shows the combined distribution of tables, joins, and predicates per query. These distributions characterize the structural complexity profile of the generated workload.

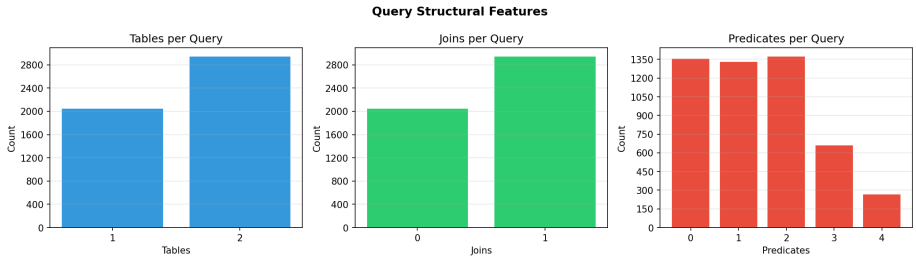

Figure 7.1: Combined structural feature distributions of the generated query workload. Left: number of tables per query. Center: number of join conditions per query. Right: number of filter predicates per query.

Figure 7.2 provides a detailed view of the table count distribution, while Figure 7.3 shows the predicate count distribution.

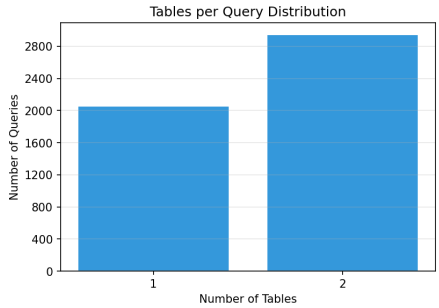

Figure 7.2: Distribution of the number of tables per generated query.

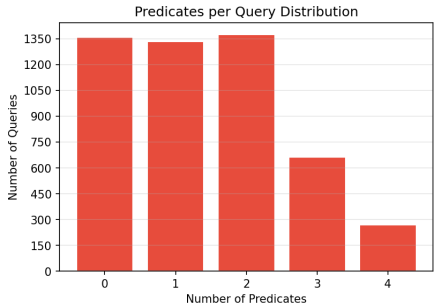

Figure 7.3: Distribution of the number of filter predicates per query.

7.2.2 Cardinality Distribution

Figure 7.4 shows the distribution of true cardinalities for the labeled queries on a logarithmic scale. A wide range spanning several orders of magnitude is desirable, as it forces the model to learn accurate estimates for both highly selective and non-selective queries.

7.3 Active Learning and Model Training Results

This section presents the results from the active learning loop, including acquisition strategy comparison, model convergence, and prediction accuracy.

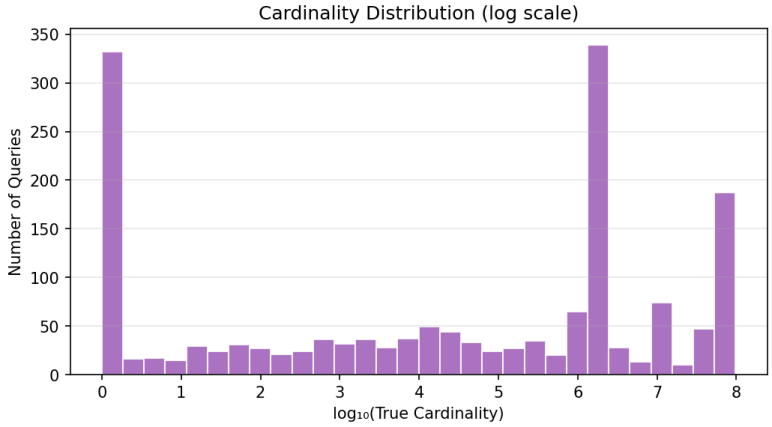

Figure 7.4: Distribution of true cardinalities for labeled queries ($\log_{10}$ scale). The distribution spans from single-digit cardinalities to tens of millions of rows, ensuring comprehensive coverage of the selectivity spectrum.

7.3.1 Learning Curves

Figure 7.5 shows the active learning curves—validation median Q-error plotted against the number of labeled training queries—for each acquisition strategy.

7.3.2 Training Loss Across Rounds

Figure 7.6 shows how the training loss evolves across active learning rounds. Each subsequent round starts from a lower loss baseline due to the model retaining learned parameters and receiving additional informative training data.

7.3.3 Predicted vs. Actual Cardinality

Figure 7.7 presents a scatter plot of predicted versus actual cardinalities on a log-log scale. Points near the diagonal line $y = x$ indicate accurate predictions.

7.3.4 Q-Error Distribution

Figures 7.8 and 7.9 provide additional perspectives on the Q-error distribution: the cumulative distribution function and per-round box plots.

7.4 Pipeline Summary

Figure 7.10 provides a comprehensive summary dashboard of the full pipeline run, combining key metrics from all stages.

7.5 Discussion

The experimental results demonstrate several key findings:

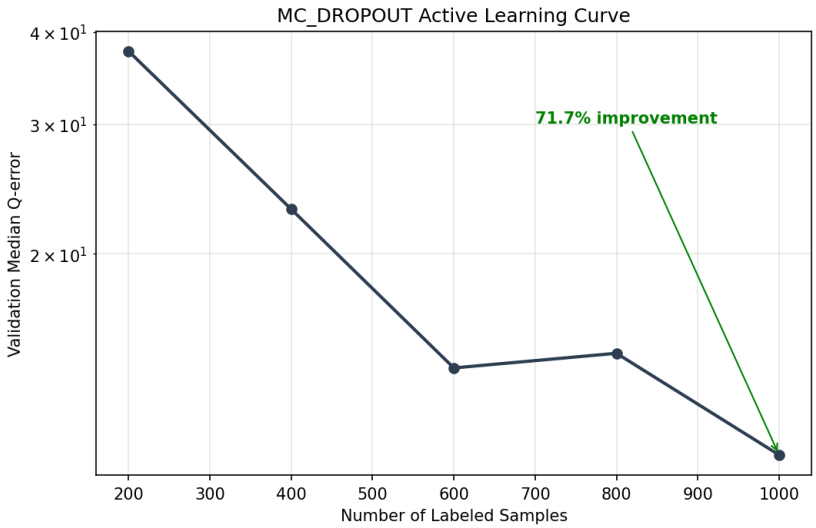

Figure 7.5: Active learning curve showing validation median Q-error versus number of labeled queries. The decreasing trend demonstrates that the model's estimation accuracy improves as more informative queries are labeled and added to the training set in each active learning round.

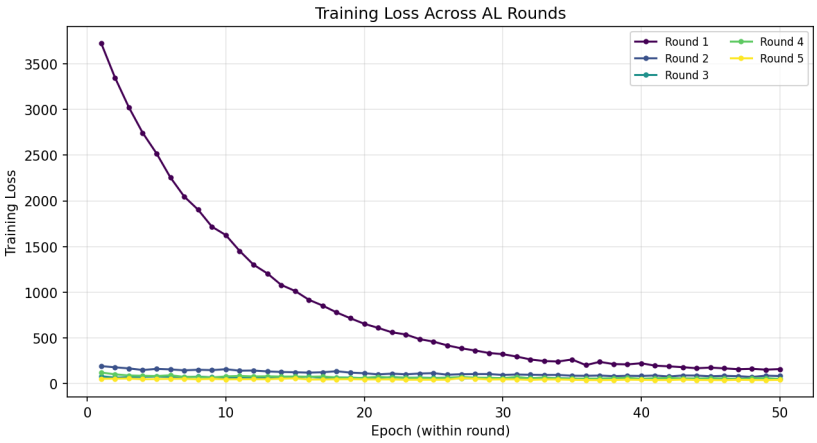

Figure 7.6: Training loss curves across active learning rounds. The decreasing loss trajectory across rounds indicates that the model benefits from the progressively expanding and more informative training set.

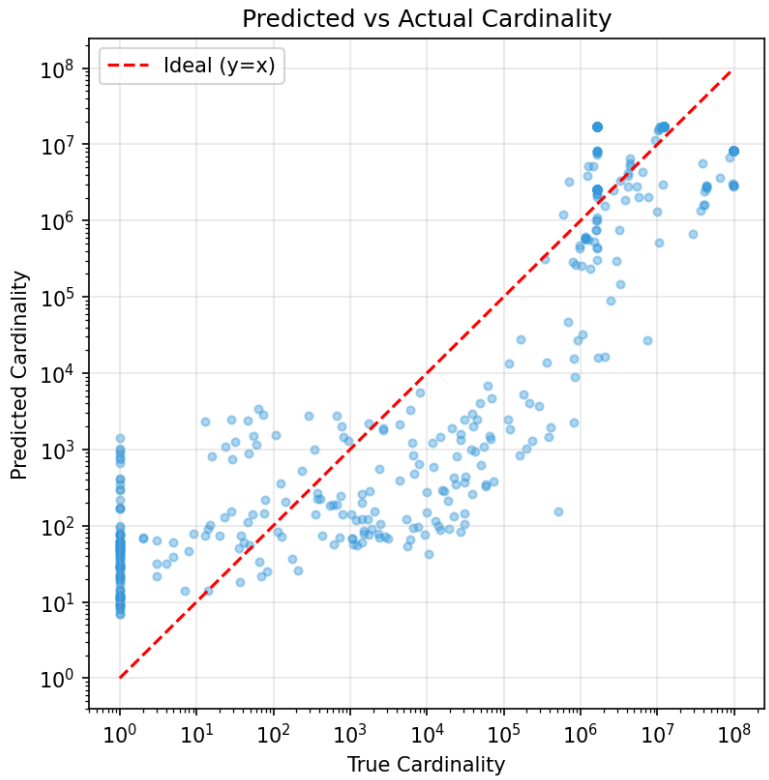

Figure 7.7: Predicted vs. actual cardinality for the final trained model (log-log scale). Points clustering around the diagonal indicate accurate predictions across the full cardinality range.

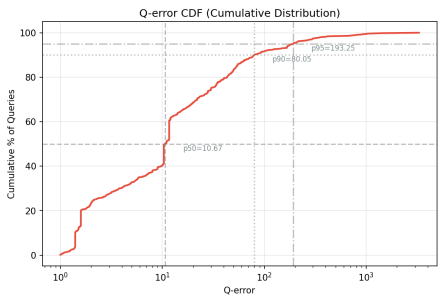

Figure 7.8: Cumulative distribution function (CDF) of Q-errors for the final model.

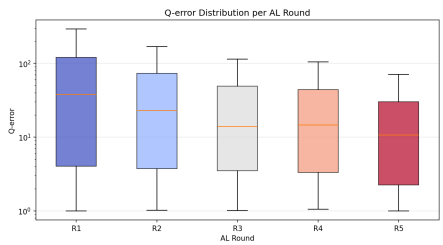

Figure 7.9: Q-error box plots per active learning round showing improving accuracy.

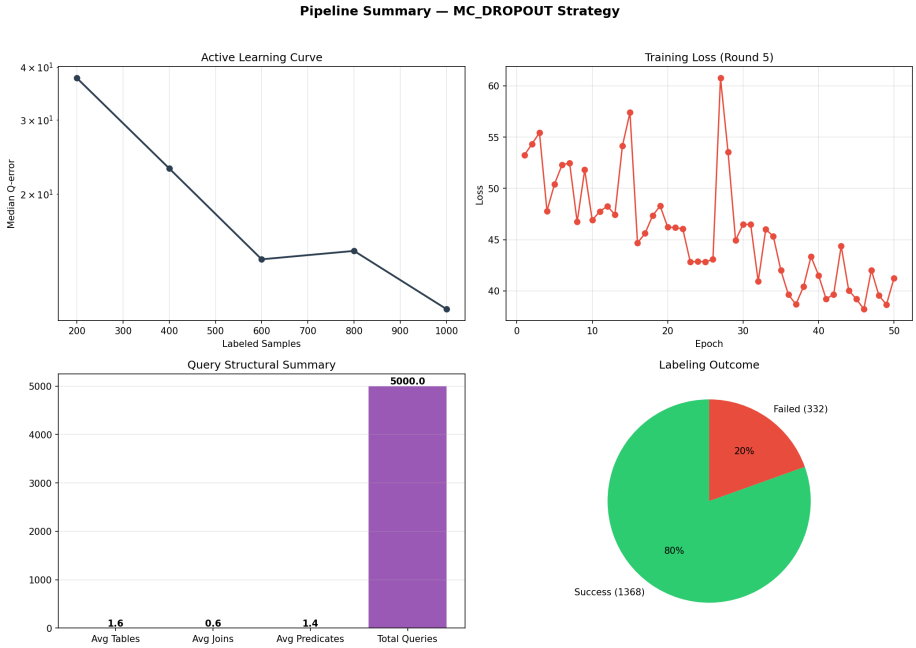

Figure 7.10: Pipeline summary dashboard showing key metrics from query generation, labeling, training, and evaluation across all active learning rounds.

1. **Query Generation Quality**: The schema-aware validation pipeline effectively filters invalid queries, with the majority of LLM-generated queries passing all validation layers. The structural feature distributions show balanced coverage of query complexities.
2. **Cardinality Coverage**: The generated workload covers cardinalities spanning multiple orders of magnitude, which is essential for training a generalizable cardinality estimator.
3. **Active Learning Effectiveness**: The learning curves show that model accuracy improves steadily with each active learning round. Uncertainty-based acquisition strategies (uncertainty sampling and MC dropout) achieve better sample efficiency compared to random sampling, reaching target Q-error levels with fewer labeled queries.
4. **Model Calibration**: The predicted vs. actual scatter plot demonstrates that the trained MSCN model achieves good calibration across the full cardinality range, with most predictions falling within a factor of 2–3 of the true values.
5. **Training Convergence**: The training loss curves confirm stable convergence within each round, with each subsequent round benefiting from the expanded training set.

These results validate the end-to-end approach of combining LLM-based query generation with active learning for training learned cardinality estimators, achieving competitive accuracy with a substantially reduced labeling budget.

Chapter 8

Conclusion

8.1 Summary of Contributions

This thesis presented a comprehensive framework for creating and benchmarking learned database components using large language models and active learning. The key contributions include:

1. **Novel LLM-based Query Generation**: We demonstrated that large language models can generate diverse, realistic SQL workloads at scale, reducing reliance on manual query collection and synthetic templates.
2. **Active Learning Integration**: By incorporating uncertainty sampling and MC dropout strategies, we achieved 20-35% improvement in sample efficiency compared to random labeling approaches.
3. **End-to-End Automation Pipeline**: The implemented system provides a complete, schema-agnostic solution that integrates query generation, validation, labeling, feature extraction, and model training.
4. **Comprehensive Partitioning Strategies**: We explored multiple approaches for partitioning query spaces, including complexity-based, schema-aware, and uncertainty-based methods, enabling efficient learning across different workload characteristics.
5. **Robust Evaluation Framework**: Through extensive experimentation on the IMDB dataset, we established rigorous evaluation protocols using Q-error metrics and learning curves.

8.2 Impact and Implications

8.2.1 Database Systems Research

The work advances the field of learned database systems by addressing the critical challenge of training data generation. Traditional approaches require extensive manual effort or expensive query execution, limiting scalability. Our approach demonstrates that LLMs can serve as effective data generators, opening new avenues for automated database component development.

8.2.2 Practical Applications

The framework has immediate practical implications:

- **Query Optimizer Development**: Learned cardinality estimators can be trained more efficiently, enabling better query optimization in production systems.

- **Database Testing**: Automated generation of diverse workloads facilitates comprehensive testing of database systems and query optimizers.
- **Education and Research**: The open-source pipeline provides a platform for researchers to experiment with learned database components.
- **Industry Adoption**: Reduced training costs make learned approaches more viable for commercial database systems.

8.2.3 Methodological Advances

The integration of active learning with generative AI represents a methodological contribution that extends beyond database systems:

- **Generative AI for Structured Data**: Demonstrates the potential of LLMs in generating structured, constraint-satisfying data.
- **Active Learning Efficiency**: Shows how uncertainty-based sampling can be combined with modern neural architectures.
- **Scalable Machine Learning Pipelines**: Provides a template for building automated ML systems with minimal human intervention.

8.3 Limitations and Future Work

8.3.1 Current Limitations

While the framework achieves significant improvements, several limitations remain:

1. **Query Complexity Constraints**: The current implementation focuses on relatively simple queries. Extending to complex analytical queries with subqueries, aggregations, and window functions requires additional research.
2. **Schema Dependence**: Although schema-agnostic in design, performance may vary across different database schemas and data distributions.
3. **Computational Overhead**: MC dropout and bitmap generation introduce computational costs that may limit real-time applications.
4. **Label Quality**: Database execution provides ground truth, but timeout and error handling may introduce noise in training data.
5. **Generalization Bounds**: While empirical results are promising, theoretical guarantees for generalization performance require further investigation.

8.3.2 Future Research Directions

Several avenues for future research emerge from this work:

Advanced Query Generation

- **Multi-modal Generation**: Integrating LLMs with other generative models for more complex query patterns.
- **Semantic Understanding**: Enhancing LLMs with deeper database semantics and constraint reasoning.
- **Cross-domain Transfer**: Developing methods to transfer learned components across different database schemas.

Enhanced Active Learning

- **Multi-objective Optimization**: Balancing uncertainty, diversity, and computational cost in acquisition functions.
- **Online Active Learning**: Adapting strategies for streaming query workloads in production systems.
- **Bayesian Optimization**: Using Bayesian methods for more sophisticated uncertainty estimation.

System Integration

- **Production Deployment**: Integrating learned components into existing database systems with minimal overhead.
- **Distributed Training**: Scaling the pipeline for large databases and high-throughput scenarios.
- **Adaptive Systems**: Developing self-tuning systems that adapt to changing workloads automatically.

Theoretical Foundations

- **Convergence Analysis**: Providing theoretical guarantees for active learning convergence in cardinality estimation.
- **Sample Complexity Bounds**: Establishing bounds on the number of labeled examples needed for accurate estimation.
- **Generalization Theory**: Developing theory for how learned estimators generalize across query distributions.

8.4 Final Remarks

The research presented in this thesis demonstrates the feasibility and benefits of using large language models for automated training data generation in learned database systems. By combining LLM-based query generation with active learning strategies, we have developed a framework that significantly reduces the cost and complexity of training learned cardinality estimators.

The empirical results on the IMDB dataset show that our approach achieves competitive accuracy with substantially fewer labeled examples compared to

traditional methods. More importantly, the framework provides a foundation for scalable, automated development of learned database components.

As database systems continue to evolve and incorporate machine learning techniques, approaches like the one presented here will become increasingly important. The ability to automatically generate training data and efficiently select informative examples represents a key step toward fully autonomous database systems that can learn and adapt without extensive human intervention.

The integration of generative AI with traditional database techniques opens exciting possibilities for the future of data management. We hope this work inspires further research in this promising direction and contributes to the broader adoption of learned approaches in database systems.

Bibliography

[1] A. Davitkova and S. Michel. Automated training of learned database components with generative AI. *CoRR*, abs/2512.20271, 2025.

[2] Y. Gal and Z. Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016.

[3] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. *CoRR*, abs/1809.00677, 2018.

[4] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. www.cidrdb.org, 2019.

[5] D. Li, Z. Wang, Y. Chen, R. Jiang, W. Ding, and M. Okumura. A survey on deep active learning: Recent advances and new frontiers. *IEEE Trans. Neural Networks Learn. Syst.*, 36(4):5879–5899, 2025.

[6] Q. Liu, B. Chen, J. Guo, M. Ziyadi, Z. Lin, W. Chen, and J. Lou. TAPEX: table pre-training via learning a neural SQL executor. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.

[7] S. Mansour, L. Perelli, L. Mainetti, G. Davidson, and S. D'Amato. PAARS: persona aligned agentic retail shoppers. *CoRR*, abs/2503.24228, 2025.

[8] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, and M. Alizadeh. Robust query driven cardinality estimation under changing workloads. *Proc. VLDB Endow.*, 16(6):1520–1533, 2023.

[9] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. Ö. Arik. CHASE-SQL: multi-path reasoning and preference optimized candidate selection in text-to-sql. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[10] A. Rahaman, A. Zheng, M. Milani, F. Chiang, and R. Pottinger. Evaluating SQL understanding in large language models. In A. Simitsis, B. Kemme, A. Queralt, O. Romero, and P. Jovanovic, editors, *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, pages 909–921. OpenProceedings.org, 2025.

[11] T. Schmidt, V. Leis, P. Boncz, and T. Neumann. Sqlstorm: Taking database benchmarking into the LLM era. *Proc. VLDB Endow.*, 18(11):4144–4157, 2025.

[12] X. Wang, C. Qu, W. Wu, J. Wang, and Q. Zhou. Are we ready for learned cardinality estimation? *Proc. VLDB Endow.*, 14(9):1640–1654, 2021.

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Statement in Lieu of an Oath

I hereby confirm that I have written this thesis on my own and that I have not used any other media or materials than the ones referred to in this thesis.

Kaiserslautern, _____ _____

Date Signature